Understanding DAX Pillars

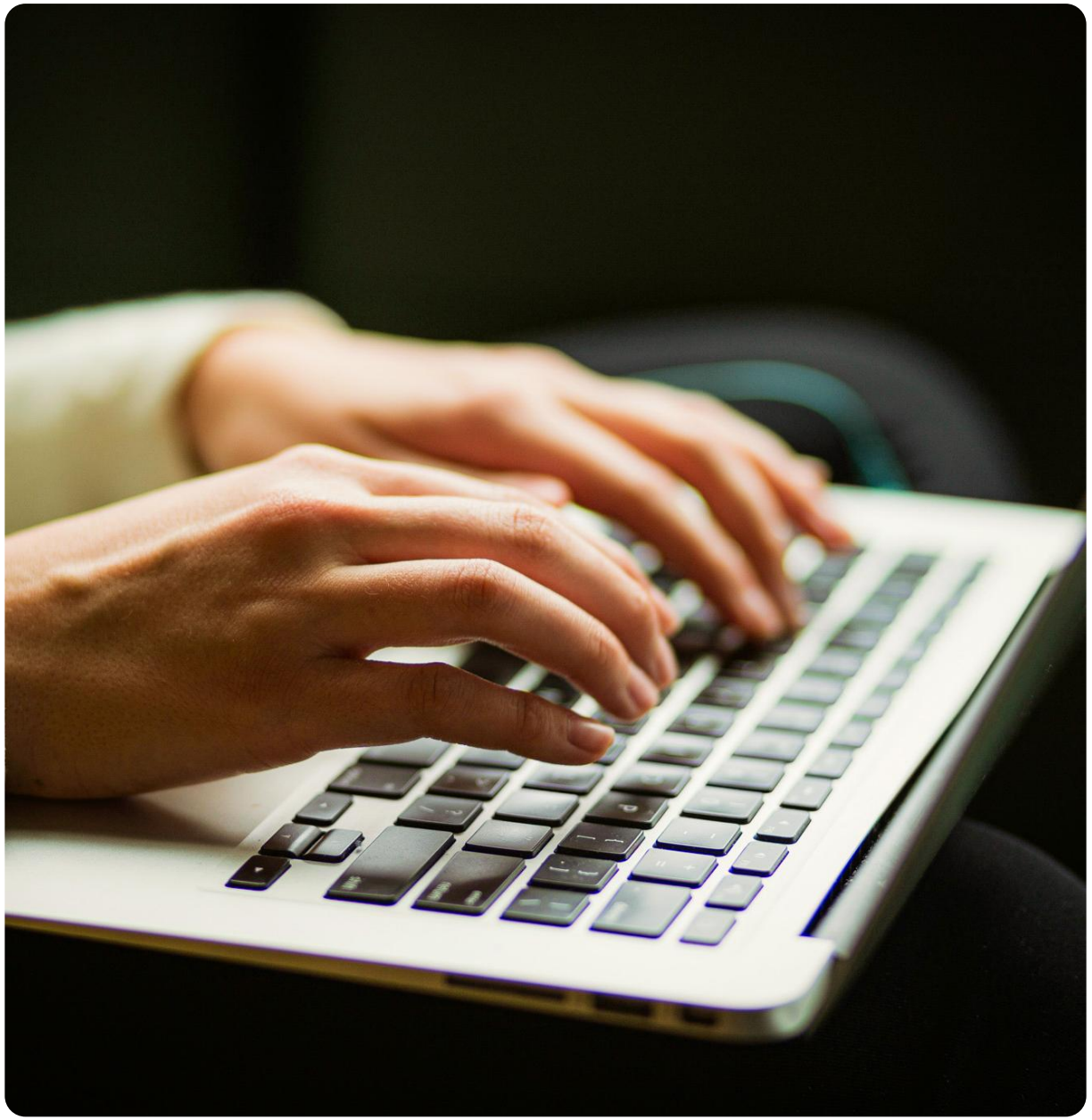
Francesco Bergamaschi
Daten-WG
Oct 14 2026

Wi-Fi: The Grid
Password: wifi-TG!

Did not Download the material?
Mail francesco.bergamaschi@unibo.it

Agenda

- Intro + Presentations (short)
- Part 1 Row Context and Calculated Columns
- Part 2 Filter Context and Measures
- *Coffee Break*
- Part 3 Iterators
- Morning Summary and Q&A
- *Lunch break (1 h)*
- Part 4 Table Functions in Iterators
- Part 5 Context Transition
- *Coffee Break*
- Part 6 Expanded Tables
- Wrap up
- Final Q&A and survey



Francesco Bergamaschi

Consultant and Professor of BI & Analytics at Bologna University, co-founder of kubisco

Passionate about teaching DAX and crunching data problems

Bologna

Loves to play drums





Fabric Super User



Kudo's Received



Answers provided



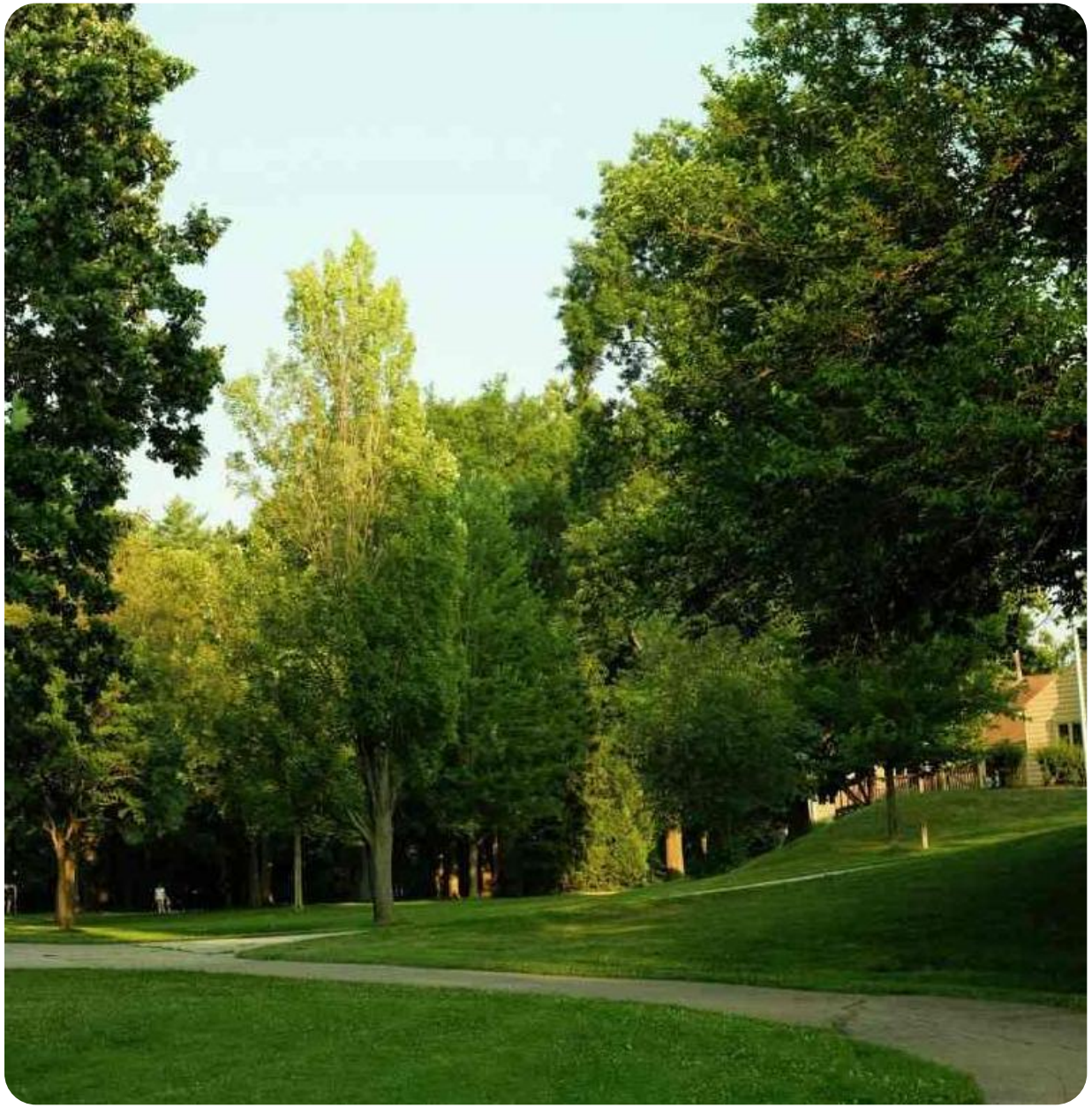
Solutions provided



Founding member

Before we begin

- I love questions, ask them as soon as they come to your mind, do not wait
- I want you to (possibly) do all I do on Power BI Desktop, anyway I shall share the final file for reference
- I understand you need breaks (so do I 😊)
- I understand this workshop involves a LOT of stuff, so I shall be helping you in the journey also after the session
- If you want to learn DAX, you must accept (at the end of the session), the following weird statements
  - 1 – SUM does not exist
  - 2 – FILTER does not *filter*
  - 3 – Variables are constant



DAX Pillars

Row Context	Filter Context	Iterators	Context Transition	Expanded Table

DAX Pillars

⇶ Row Context

Row context iterates a table (it does not filter it natively).

When you add a column to a table, the row context lets a single row's calculation use other values *in the same row*.

It does not propagate automatically to other tables. If you want to propagate this, use RELATED or RELATEDTABLE.

A column reference in a DAX expression requires a row context to return the value of the column from a table.

Row Context is iterating "the visible" table row by row. We shall come back on who decides which are the visible rows.

The row context is native in Calculated Columns in DAX.

Filter can be injected in calculated columns through CALCULATE (context transition)

Filter Context

Filter context filters the data model

When you "ask for a table", the filter context is the set of filters that gets applied before the table is handed over to you.

Propagates automatically if allowed by direction of relationship

Report-, Page- and Visual Level Filters, slicers, visual interactions and within visual → Every data point is calculated within its own filter context

You can use the CALCULATE DAX function to modify filter context and then calculate a number under the new filter context with overwrite, remove or add

Filter context is evaluated first before calculating the value in a CALCULATE

A filter is a table. A table is a list of values. The filter context is the set of active filters.

Native in measures

Iterators

Through iterators, you can create a row context in a measure (iterate a table) or a nested row context in a calculated column
Iterations do not filter and filters do not iterate: Iterators iterate (create a row context) on the visible rows of a table.

Who decides which rows are visible so they can be iterated? The filter context.

So, the filter context filters the model first and sets the rows available for each calculation, the row context iterates those rows.

⇆ Context Transition

There is no filter context in a row context. Unless you specify a context transition.

Context transition is the process of converting a row context to an equivalent filter context

Use CALCULATE or a Measure Reference in a row-context to enable context transition.

Every measure reference always has an implicit CALCULATE surrounding it

Context transition is expensive operation. If used during an iteration on a table with 10 columns and one million rows, then CALCULATE needs to apply 10 filters, one million times.

Context transition does not only filter one row

Context transition uses columns that are not present in the formular

Calculated tables and columns aren't evaluated within filter context

Expanded Table

When you ask for a table, you get the expanded table

Looks like the One Big Table (OBT) as it joins the related table columns

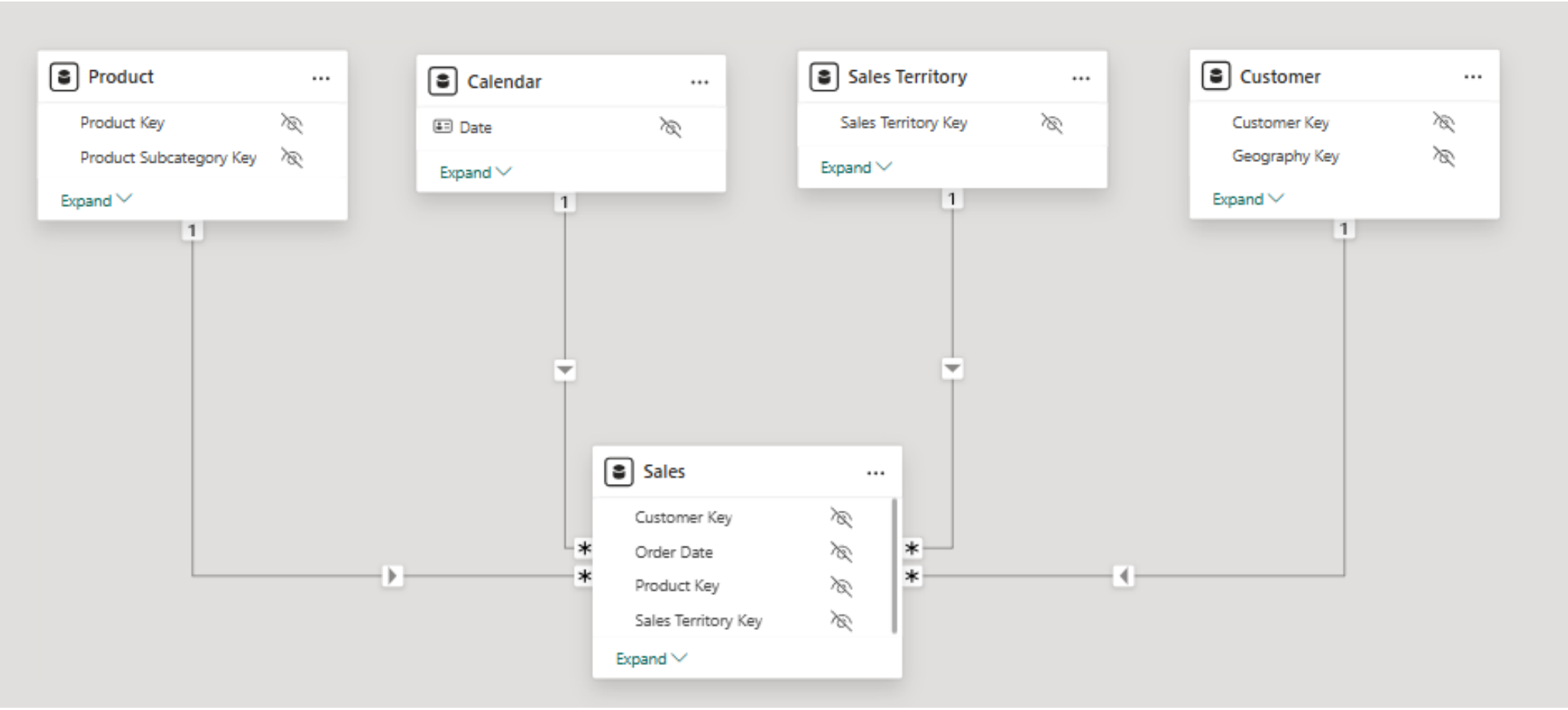
If you refer to the fact table of Sales, it will refer to all native columns in the Fact table + all related columns from the dimension tables that have a one-to-many relationship to the fact table

For performance optimization, always focus on only including the needed columns. Example of bad practice:

```
SUMX (
    FILTER (
        'Fact Table',
         Expression
    )
)
```

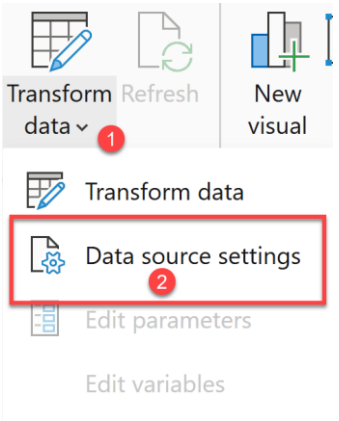
"Never filter tables, always filter columns"
- SQLBI

What will we work on today?



Power BI Desktop settings and redirection to the data source

- No automatic date/time
- On-Object Interaction (you will need to restart Power BI Desktop)
- Data source redirection to our data source



DAX Notation

- Sales[Unit price] (mandatory for Calculated Columns, avoid [UnitPrice])
- Sales (table)
- [Revenues] (mandatory for Measures, avoid Sales[Revenues])
- _Sales (suggested for Variables)
- [@Column] for columns created on the fly

Row Context and Calculated Columns

Important points to learn in this section:

- Purpose of Calculated Columns
- Calculated Columns DAX code’s evaluation context rules (row context and empty filter context)
- Iterating is not filtering

Purpose of Calculated Columns

- Grouping
- Filtering
- Be a key for a relationship
- Support a slow measure (that should be a rare case after you complete this Masterclass ☺)

Row Context and Calculated Columns

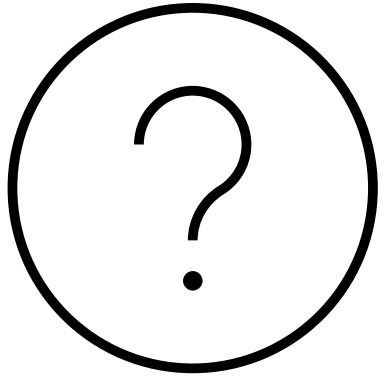
- Consider the following DAX espression, can you solve it?

  IF (Sales[UnitPrice] < 1000, “low”, “high”), let’s see it visually

- IF

UnitPrice
$3.578,27
$3.399,99
$699,10
$3.399,99
$3.399,99
$3.374,99
$3.578,27
$3.578,27
$3.399,99
$699,10
$3.578,27
$3.578,27
$3.578,27

< 1000, “low”, “high”)



Row Context and Calculated Columns

- We cannot solve it, how can we compare on value (1.000) with multiple values (the different values of *Sales[UnitPrice]*? Or can we?

IF (Sales[UnitPrice] < 1000, “low”, “high”)

- IF

UnitPrice
$3.578,27
$3.399,99
$699,10
$3.399,99
$3.399,99
$3.374,99
$3.578,27
$3.578,27
$3.399,99
$699,10
$3.578,27
$3.578,27
$3.578,27

< 1000, “low”, “high”)

Row Context and Calculated Columns

- Suppose now we try to solve the below code on row at a time, would we be able get something out?

```
IF ( Sales[UnitPrice] < 1000, “low”, “high” )
```

- IF 

< 1000, “low”, “high”) → result: high

Row Context and Calculated Columns

- Let's do the thing on each row, here is the result

  IF (Sales[UnitPrice] < 1000, "low", "high")

- So what can we conclude?

UnitPrice	Test
$3.578,27	high
$3.399,99	high
$699,10	low
$3.399,99	high
$3.399,99	high
$3.374,99	high
$3.578,27	high
$3.578,27	high
$3.399,99	high
$699,10	low
$3.578,27	high
$3.578,27	high
$3.578,27	high
$3.578,27	high
$3.578,27	high
$3.578,27	high
$3.578,27	high

Row Context and Calculated Columns

- So what can we conclude?
- That when we reference a column in DAX, **this can have a meaning only if we go row by row on it.**
- The meaning of

  IF (Sales[UnitPrice] < 1000, “low”, “high”)

  is:

  if the value of the column Sales[UnitPrice] in the current row if lower than 1000, then return “low”, otherwise return “high”

UnitPrice	Test
$3.578,27	high
$3.399,99	high
$699,10	low
$3.399,99	high
$3.399,99	high
$3.374,99	high
$3.578,27	high
$3.578,27	high
$3.399,99	high
$699,10	low
$3.578,27	high
$3.578,27	high
$3.578,27	high
$3.578,27	high
$3.578,27	high
$3.578,27	high
$3.578,27	high

Row Context and Calculated Columns

- So what *else* can we conclude?
- That we have created a *calculated column on the table the column belongs to (Sales)*, we have *no other way* to solve that DAX expression
- We define ***row context*** the presence of a current row in a process of scanning a set of rows one row at a time
- Everytime you create a calculated column you do

  it *on a table* and, in order to solve the DAX code,

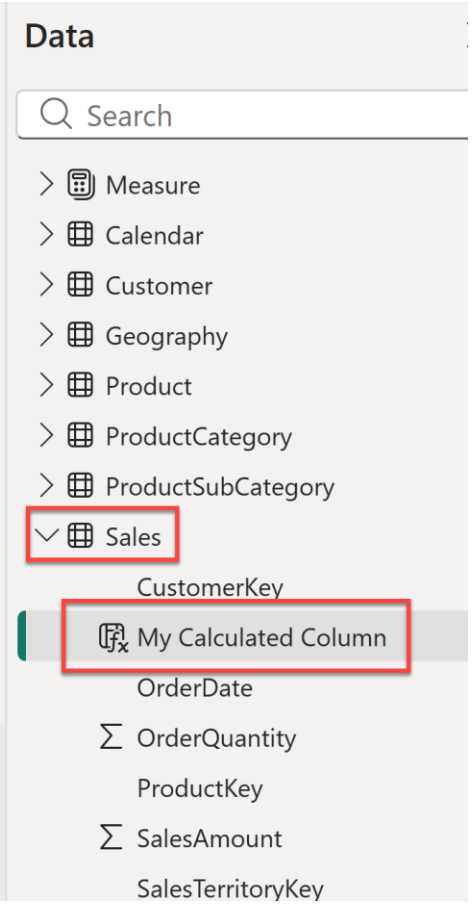
  a row context is automatically created on the table

UnitPrice	Test
$3.578,27	high
$3.399,99	high
$699,10	low
$3.399,99	high
$3.399,99	high
$3.374,99	high
$3.578,27	high
$3.578,27	high
$3.399,99	high
$699,10	low
$3.578,27	high
$3.578,27	high
$3.578,27	high
$3.578,27	high
$3.578,27	high
$3.578,27	high
$3.578,27	high

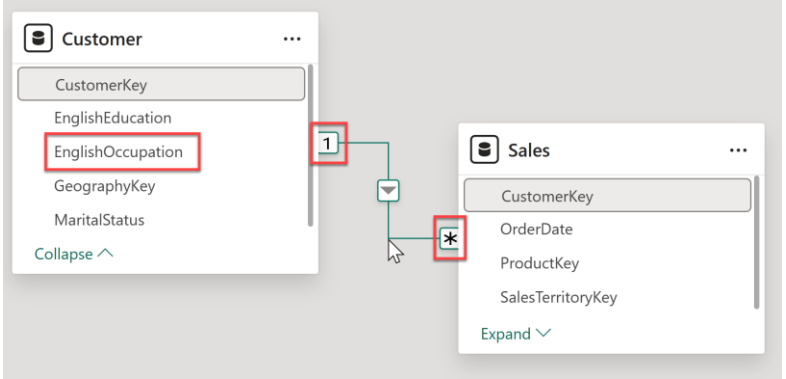
Row Context and Calculated Columns

```
1 My Calculated Column =
2 -- here a row context on the Sales table is created by
  Power BI Desktop automatically, as this is a calulated
  column so we can refer to any value in the current row
  and use it
3 IF (
4     Sales[OrderQuantity] < 3, "standard qty", "high qty"
5 )
```

OrderDate	ShipDateKey	CustomerKey	SalesTerritoryKey	OrderQuantity	UnitPrice	SalesAmount	My Calculated Column
20/lug/2004	20040626	14038	9	3	$2.384,07	$7.152	high qty
08/lug/2004	20040614	14039	9	2	$2.384,07	$4.768	standard qty
28/lug/2004	20040704	14045	9	2	$2.384,07	$4.768	standard qty
10/lug/2004	20040616	15566	6	3	$2.384,07	$7.152	high qty



Row Context and Calculated Columns



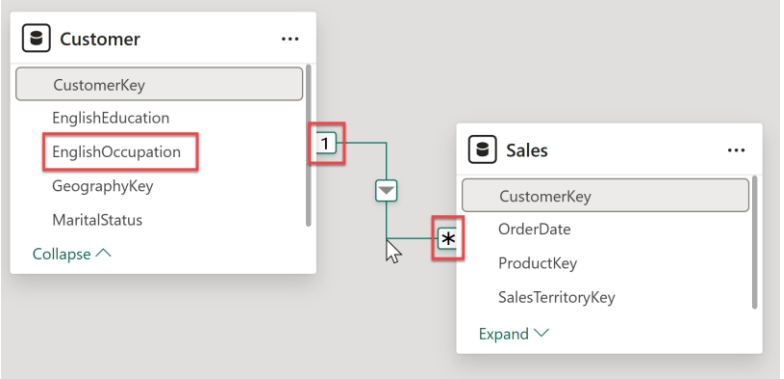
- How about columns in a table connected 1 to many to the one on which I am creating a calculated column? Example: **another calculated column in the *Sales* table**
- Can I reference the columns of this table in my calculated column?
- Not directly:

```
1 Test Customer Table =
2 Customer[EnglishEducation]
```

A single value for column 'EnglishEducation' in table 'Customer' cannot be determined. This can happen when a measure or function formula refers to a column that contains many valu

Row Context and Calculated Columns

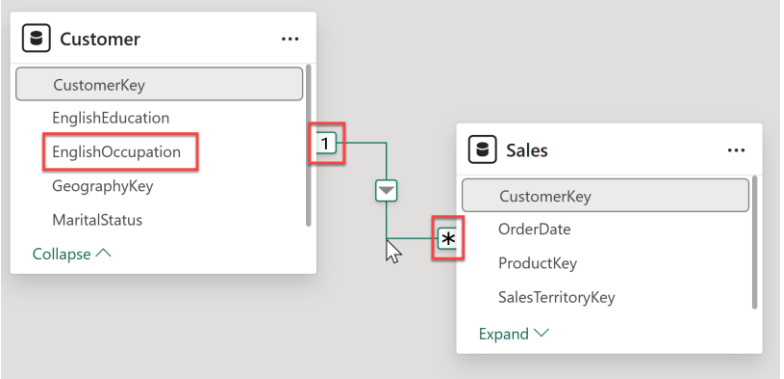
- But you can if you specify a keyword, *RELATED*




```
1 Test Customer Table =
2 IF (
3     RELATED( Customer[EnglishEducation] ) = "Bachelors",
4     "Bach Cust Transation",
5     "Not Bach Cust Transaction"
6 )
```

esTerritoryKey	OrderQuantity	UnitPrice	SalesAmount	My Calculated Column	Another Calculated Column	Test Customer Table
4	1	$4,99	$5	standard qty	63.163	Not Bach Cust Transaction
4	1	$4,99	$5	standard qty	63.163	Bach Cust Transation
4	1	$4,99	$5	standard qty	63.163	Not Bach Cust Transaction
4	1	$4,99	$5	standard qty	63.163	Not Bach Cust Transaction
4	1	$4,99	$5	standard qty	63.163	Bach Cust Transation
4	1	$4,99	$5	standard qty	63.163	Not Bach Cust Transaction

Row Context and Calculated Columns

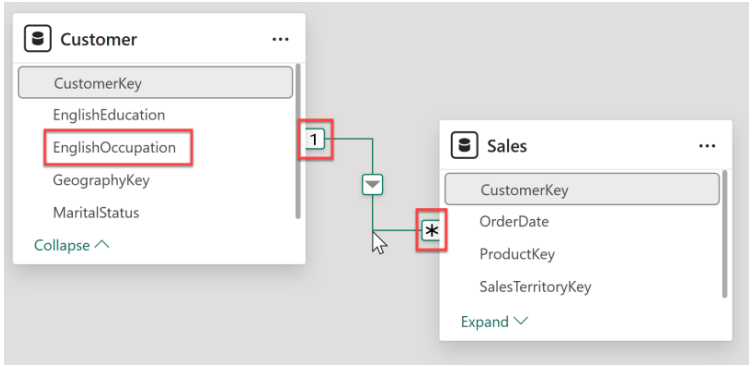


- Important note:

  RELATED is able to fetch any value from any column of a table connected 1 to many to the one on which we are creating the calculated column

- This is because that is a *single* value since only one row exists on the one-side table corresponding to the currently iterated (row context) row on the many side

Row Context and Calculated Columns



- Important note:

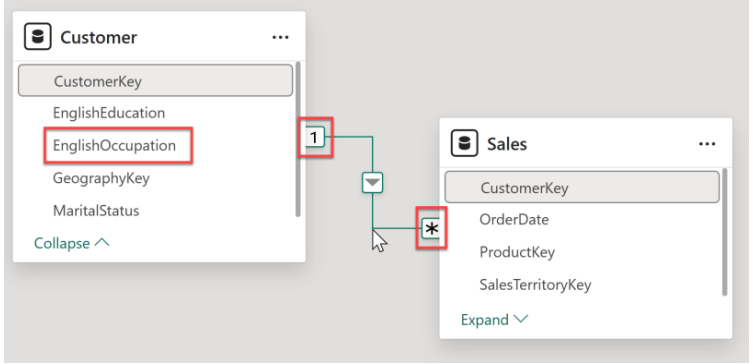
  In the opposite scenario *RELATED* cannot work as there would be a *set* of rows, and therefore of values. Example: a calculated column on the *Customer* table

```
1 Test Sales Table =
2 RELATED ( Sales[OrderDate] )
```

The column 'Sales[OrderDate]' either doesn't exist or doesn't have a relationship to any table available in the current context.

GeographyKey	MaritalStatus	EnglishEducation	EnglishOccupation	Test Sales Table
37	S	Bachelors	Professional	#ERROR
11	S	Bachelors	Professional	#ERROR
19	S	Bachelors	Professional	#ERROR
22	S	Bachelors	Professional	#ERROR

Row Context and Calculated Columns



- Important note:

  RELATEDTABLE is your friend in this scenario

```
1 Test Sales Table =
2 COUNTROWS ( RELATEDTABLE( Sales ) )
```

	GeographyKey	MaritalStatus	EnglishEducation	EnglishOccupation	Test Sales Table
)01	37	S	Bachelors	Professional	11
)03	11	S	Bachelors	Professional	9
)04	19	S	Bachelors	Professional	6
)05	22	S	Bachelors	Professional	6

  but: 1- we cannot cite a single column but the entire table as *RELATEDTABLE* returns entire rows

  2- **we need to aggregate its result in some way.** In fact:

  the DAX code of a calculated column must return a single value (scalar expression)

- Side note: *RELATEDTABLE* is an alias for *CALCULATETABLE* (further details later on)

Row Context and Calculated Columns

The DAX code of a calculated column must return a single value (scalar expression)

Weird, right? Let’s discuss this important statement

Row Context and Calculated Columns



- Consider now the following DAX expression in a calculated column in the Sales table, what do you expect as a result?

```
1 Another Calculated Column =
2 COUNTROWS( Sales )
```

CustomerKey	SalesTerritoryKey	OrderQuantity	UnitPrice	SalesAmount	My Calculated Column	Another Calculated Column
11082	4	1	$4,99	$5	standard qty	
11088	4	1	$4,99	$5	standard qty	
11135	4	1	$4,99	$5	standard qty	
11161	4	1	$4,99	$5	standard qty	
11171	4	1	$4,99	$5	standard qty	
11175	4	1	$4,99	$5	standard qty	
11208	4	1	$4,99	$5	standard qty	
11227	4	1	$4,99	$5	standard qty	
11252	4	1	$4,99	$5	standard qty	
11267	4	1	$4,99	$5	standard qty	
11273	4	1	$4,99	$5	standard qty	
11281	4	1	$4,99	$5	standard qty	
11292	4	1	$4,99	$5	standard qty	
11298	4	1	$4,99	$5	standard qty	

Row Context and Calculated Columns



- You might expect that, since the expression is evaluated one row at a time, only one row at a time will be *visible* (so you expect 1 on every row)

```
1 Another Calculated Column =
2 COUNTROWS( Sales )
```

CustomerKey	SalesTerritoryKey	OrderQuantity	UnitPrice	SalesAmount	My Calculated Column	Another Calculated Column
11082	4	1	$4,99	$5	standard qty	
11088	4	1	$4,99	$5	standard qty	
11135	4	1	$4,99	$5	standard qty	
11161	4	1	$4,99	$5	standard qty	
11171	4	1	$4,99	$5	standard qty	
11175	4	1	$4,99	$5	standard qty	
11208	4	1	$4,99	$5	standard qty	
11227	4	1	$4,99	$5	standard qty	
11252	4	1	$4,99	$5	standard qty	
11267	4	1	$4,99	$5	standard qty	
11273	4	1	$4,99	$5	standard qty	
11281	4	1	$4,99	$5	standard qty	
11292	4	1	$4,99	$5	standard qty	
11298	4	1	$4,99	$5	standard qty	

Row Context and Calculated Columns



- That's what I expected when I was learning DAX ☺ but the result is very, very different: the same number on each row but not 1 but 63.163, what nr is it?

```
1 Another Calculated Column =
2 COUNTROWS( Sales )
```

CustomerKey	SalesTerritoryKey	OrderQuantity	UnitPrice	SalesAmount	My Calcu
11082	4	1	$4,99	$5	standard
11088	4	1	$4,99	$5	standard
11135	4	1	$4,99	$5	standard
11161	4	1	$4,99	$5	standard
11171	4	1	$4,99	$5	standard
11175	4	1	$4,99	$5	standard
11208	4	1	$4,99	$5	standard
11227	4	1	$4,99	$5	standard
11252	4	1	$4,99	$5	standard
11267	4	1	$4,99	$5	standard
11273	4	1	$4,99	$5	standard
11281	4	1	$4,99	$5	standard

Another Calculated Column
63.163
63.163
63.163
63.163
63.163
63.163
63.163
63.163
63.163
63.163
63.163

tegory
bCategory
r Calculated Column
erKey
culated Column
ate

Row Context and Calculated Columns



- That’s what I expected when I was learning DAX ☺ but the result is very, very different: the same number on each row but not 1 but 63.163, what nr is it?

Table: Sales (63.163 rows)

- That’s the total number of rows of the Sales table itself! Why do we get this result on every row and not 1 on every row?

Row Context and Calculated Columns



- Why do we get this result on every row and not 1 on every row?

- The reason is the following: the expression is absolutely evaluated one row at a time on Sales, since this is a calculated column on Sales, but **this does NOT imply that only one row at a time will be *visible, what it DOES imply is that only one row at a time is the currently iterated one* (or the current one to be short)**

- Understanding this point is of paramount importance

Row Context and Calculated Columns

```
1 Another Calculated Column =
2 COUNTROWS( Sales )
```

	CustomerKey	SalesTerritoryKey	OrderQuantity	UnitPrice	SalesAmount	My Calculated Column	Another Calculated Column
16	11082	4	1	$4,99	$5	standard qty	63.163
22	11088	4	1	$4,99	$5	standard qty	63.163
27	11135	4	1	$4,99	$5	standard qty	63.163
10	11161	4	1	$4,99	$5	standard qty	63.163
17	11171	4	1	$4,99	$5	standard qty	63.163
13	11175	4	1	$4,99	$5	standard qty	63.163
12	11208	4	1	$4,99	$5	standard qty	63.163
21	11227	4	1	$4,99	$5	standard qty	63.163
27	11252	4	1	$4,99	$5	standard qty	63.163
18	11267	4	1	$4,99	$5	standard qty	63.163
22	11273	4	1	$4,99	$5	standard qty	63.163

When we are on one row (the row context), only that one is the currently iterated one, so we can use the values on that row of every column, but ALL other rows are still *visible* and the result we observe should certify this

Row Context and Calculated Columns



```
1 Another Calculated Column =
2 COUNTROWS( Sales )
```

CustomerKey	SalesTerritoryKey	OrderQuantity	UnitPrice	SalesAmount	My Calculated Column	Another Calculated Column
11082	4	1	$4,99	$5	standard qty	63.163
11088	4	1	$4,99	$5	standard qty	63.163
11135	4	1	$4,99	$5	standard qty	63.163
11161	4	1	$4,99	$5	standard qty	63.163
11171	4	1	$4,99	$5	standard qty	63.163
11175	4	1	$4,99	$5	standard qty	63.163
11208	4	1	$4,99	$5	standard qty	63.163
11227	4	1	$4,99	$5	standard qty	63.163
11252	4	1	$4,99	$5	standard qty	63.163
11267	4	1	$4,99	$5	standard qty	63.163
11273	4	1	$4,99	$5	standard qty	63.163

Iterated row, this represents what row context does



```
1 Another Calculated Column =
2 COUNTROWS( Sales )
```

CustomerKey	SalesTerritoryKey	OrderQuantity	UnitPrice	SalesAmount	My Calculated Column	Another Calculated Column
11161	4	1	$4,99	$5	standard qty	63.163

Filtered row, this does NOT represent what row context does (we'll see this is what filter context does)



Row Context and Calculated Columns



- Final example: consider now the following DAX expression in a calculated column in the *Customer* table, what do you expect as a result?

```
1 Test Sales Table no RELATEDTABLE =
2 COUNTROWS( Sales )
```

	GeographyKey	MaritalStatus	EnglishEducation	EnglishOccupation	Test Sales Table	Test Sales Table no RELATEDTABLE
)01	37	S	Bachelors	Professional	11	
)03	11	S	Bachelors	Professional	9	
)04	19	S	Bachelors	Professional	6	

Row Context and Calculated Columns



- Final example: consider now the following DAX expression in a calculated column in the *Customer* table, what do you expect as a result?

```
1 Test Sales Table no RELATEDTABLE =
2 COUNTROWS( Sales )
```

	GeographyKey	MaritalStatus	EnglishEducation	EnglishOccupation	Test Sales Table	Test Sales Table no RELATEDTABLE
)01	37	S	Bachelors	Professional	11	63.163
)03	11	S	Bachelors	Professional	9	63.163
)04	19	S	Bachelors	Professional	6	63.163
)05	22	S	Bachelors	Professional	6	63.163
)06	8	S	Bachelors	Professional	5	63.163

- Let's discuss this result (in terms of "filters")

Row Context and Calculated Columns - Summary

- Row context iterates a table
- When you add a column to a table, the row context lets a single row's calculation look at other values *in the same row*
- Row context does not propagate automatically to other tables
- If you want to propagate it, use *RELATED* or *RELATEDTABLE* (this last one an alias for *CALCULATETABLE*)
- A column reference in a DAX expression requires a row context to return the value of the column from a table.
- Row Context is iterating "the visible" table row by row
- Native in Calculated Columns in DAX
- Calculated columns DAX code must return a single value (scalar expression) and their code is always evaluated on a row context automatically created on the table on which you create the calculated column
- Calculated columns are static calculations, they are calculated only at creation and at refresh time
- Iterating is NOT filtering, so a row context has nothing to do with filters: iterating a row in the *Customer* table does not mean filtering that row. Furthermore, filters are not involved in calculated columns natively. We shall be able to inject filters in calculated columns with *CALCULATE*

Row Context – Final Note

- Can I start a row context of my own?

- Yes, either nesting a new row context in an existing one (advanced subject we shall not go through) or creating one from scratch (when there is none active).

- Create one from scratch (when there is none active), by definition, is possible only out of calculated columns, because in a calculated column a row context MUST be there and it is automatically created and you cannot avoid that: the place where we can think about this is **measures**

Row Context and Calculated Columns - Summary

- Row context iterates a table
- When you add a column to a table, the row context lets a single row's calculation look at other values *in the same row*
- Row context does not propagate automatically to other tables
- If you want to propagate it, use *RELATED* or *RELATEDTABLE* (this last one an alias for *CALCULATETABLE*)
- A column reference in a DAX expression requires a row context to return the value of the column from a table.
- Row Context is iterating "the visible" table row by row
- Native in Calculated Columns in DAX
- Calculated columns DAX code must return a single value (scalar expression) and their code is always evaluated on a row context automatically created on the table on which you create the calculated column
- Calculated columns are static calculations, they are calculated only at creation and at refresh time
- Iterating is NOT filtering, so a row context has nothing to do with filters: iterating a row in the *Customer* table does not mean filtering that row. Furthermore, filters are not involved in calculated columns natively. We shall be able to inject filters in calculated columns with *CALCULATE*

Row Context – Final Note

- Can I start a row context of my own?

- Yes, either nesting a new row context in an existing one (advanced subject we shall not go through) or creating one from scratch (when there is none active).

- Create one from scratch (when there is none active), by definition, is possible only out of calculated columns, because in a calculated column a row context MUST be there and it is automatically created and you cannot avoid that: the place where we can think about this is **measures**

Coffee Break

Filter Context and Measures

- Filter context filters the data model
- When you “ask for a table”, the filter context is the set of filters that gets applied before the table is handed over to you.
- Propagates automatically if allowed by direction of relationship
- Report-, Page- and Visual Level Filters, slicers, visual interactions and within visual → Every data point is calculated within its own filter context
- You can use the CALCULATE DAX function to modify filter context and then calculate a number under the new filter context with overwrite, remove or add
- Filter context is evaluated first before calculating the value in a CALCULATE
- A filter is a table. A table is a list of values. The filter context is the set of active filters.
- Native in measures

Filter Context and Measures

Important points to learn in this section:

- Purpose of Measures
- Measures' DAX code's evaluation context rules (filter context and no row context)
- Filtering is not iterating

Purpose of Measures

- Create dynamic calculations
- Quantify performance indicators

Filer Context and Measures

- Measures have no row context automatically created (on which table would that automatically happen, by the way??)
- They are, on the contrary, influenced by the filter context
- So they are the opposite of calculated columns that have an automatic row context and are not influenced by filter context and that are static calculations

Filer Context and Measures

- Measures have no row context automatically created (on which table would that automatically happen, by the way??)

```
1 My Measure =
2 -- here no row context on the Sales table is created by Power BI Desktop automatically, as
  this is a measure so we cannot refer to any value in the current row and use it
3 IF (
4     Sales[OrderQuantity] < 3, "standard qty", "high qty"
5 )
```

⚠ A single value for column 'OrderQuantity' in table 'Sales' cannot be determined. This can happen when a measure or function formula refers to a column that contains many va

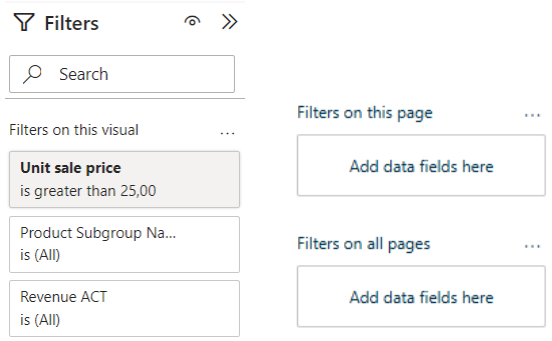
- So in a measure, subject to filters, no iteration is there. Filters are.

Filter Context

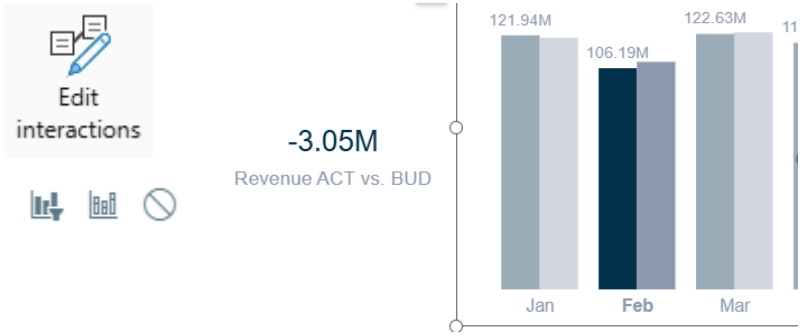
- A filter is a set of values for one or more columns
- A filter context is a set of filters
- Where do filters that constitute the filter context come from?
  - ✓ Visual / Page / Report filters
  - ✓ Other visuals interactions (slicers or matrixes or any other visual)
  - ✓ Visual grouped columns
  - ✓ (later we shall also see) DAX code through CALCULATE!

Where filters come from (UI only for now)

Report-, Page- and Visual Level Filters



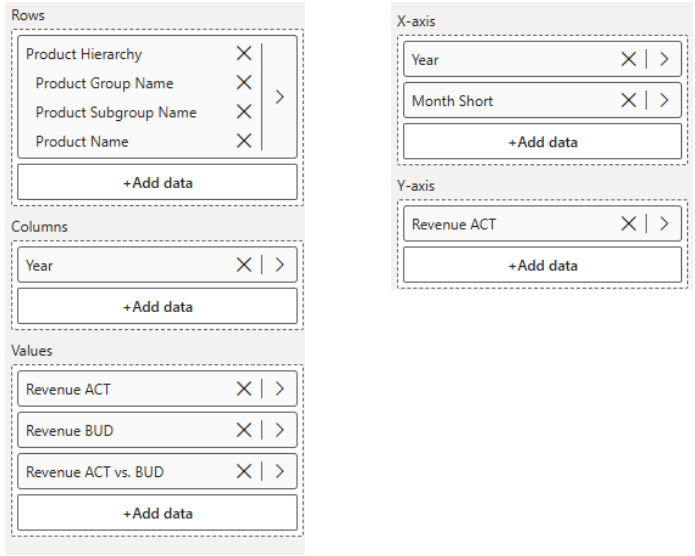
Other Visual Interactions



Slicers



Within Visuals



Dump filter in DAX Studio for filter footer context

Calendar Year	AU	CA	DE	FR	GB	US	Total
2022	$1,330,314	$154,685	$238,484	$187,728	$305,344	$1,129,671	$3,346,226
Black	$179,815	$20,370	$29,218	$23,745	$27,940	$76,250	$357,339
Red	$1,014,499	$130,915	$188,866	$153,783	$233,204	$975,221	$2,696,489
Silver	$136,000	$3,400	$20,400	$10,200	$44,200	$78,200	$292,399
2023	$2,172,188	$617,977	$529,671	$510,750	$589,799	$2,107,899	$6,528,285
Black	$643,871	$116,139	$136,628	$169,933	$196,169	$489,050	$1,751,789
Red	$1,192,662	$463,663	$311,965	$283,610	$294,848	$1,354,657	$3,901,405
Silver	$304,642	$30,171	$65,071	$39,200	$72,771	$214,171	$726,026
Yellow	$31,014	$8,004	$16,007	$18,008	$26,011	$50,022	$149,065
2024	$2,988,261	$505,839	$1,045,730	$1,016,110	$1,278,780	$2,769,766	$9,604,486
Black	$1,296,373	$212,923	$365,202	$426,512	$518,431	$1,037,924	$3,857,366
Total	$9,431,243	$2,073,160	$3,101,062	$2,818,413	$3,670,729	$10,140,215	$31,234,822

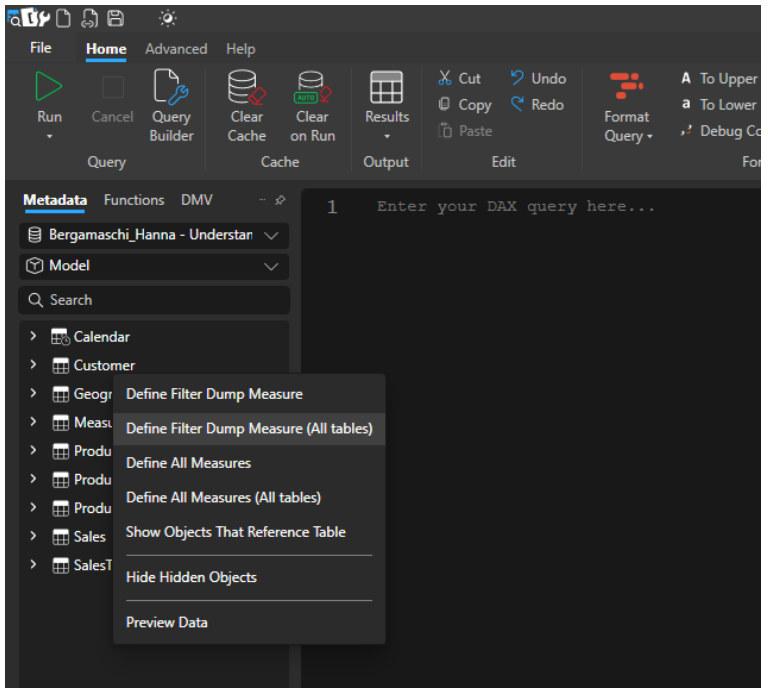
English Product Category Name

- [ ] (Blank)
- [ ] Accessories
- [x] Bikes
- [x] Clothing
- [ ] Components

DumpFilters

```
'Calendar'[Calendar Year] = 2023
Geography[Country Region Code] = DE
'Product'[Color] = Red
'Product Category'[English Product Category Name] = Bikes, Clothing
```

https://daxstudio.org/



https://www.sqlbi.com/articles/displaying-filter-context-in-power-bi-tooltips/

Measures

- Again on our measures, renamed My measure as Nr Sales Rows

```
1 Nr Sales Rows =
2 COUNTROWS( Sales )
```

Color	Nr Sales Rows	Sales Value
Black	10.212	€ 9.469.519
Blue	4.144	€ 2.570.766
Multi	4.115	€ 112.024
NA	30.396	€ 457.964
Red	5.029	€ 7.727.130
Silver	3.610	€ 5.820.151
White	593	€ 5.331
Yellow	5.064	€ 5.806.442
Total	**63.163**	**€ 31.969.327**

```
1 Sales Value =
2 SUM ( Sales[SalesAmount] )
```

- Question 1: how many rows does the Sales table have?
- Question 2: can I answer a question like “What is the result of the measure Nr Sales Rows”?
- Question 3: can I answer a question like “What is the result of the measure Nr Sales Rows when there is only a filter on the Product[Color] column, with value “Black?”
- Question 4: can I answer a question like “What is the result of the measure Nr Sales Rows when there is no filter at all?”

Filter Context and Measures - Summary

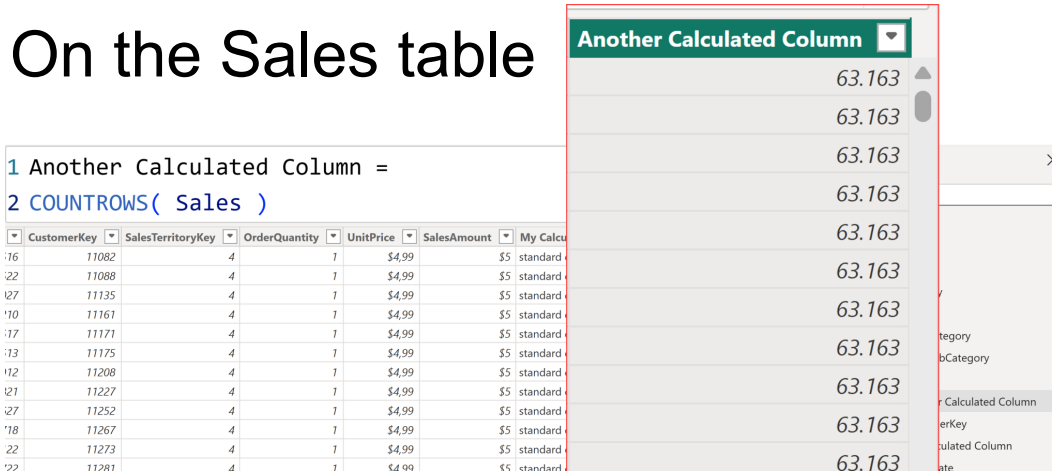
- Filter context filters the data model. Filter context is applied before executing the DAX code of a measure. A filter is a list of values from one or more columns. The filter context is a set of filters.
- When you reference a table, the filter context is the set of filters applied before the table is handed over to you.
- Propagates automatically if allowed by direction of relationship
- Report-, Page- and Visual Level Filters, slicers, visual interactions and within visual → Every data point is calculated within its own filter context
- Native in measures
- Measures DAX code must return a single value (scalar expression) and their code is always evaluated on a filter context automatically created by grouping columns in a visual, slicer selections, visual- page and report-filters and/or by other visuals interactions
- Measures are dynamic calculations, they are always calculated on the fly
- Filtering is NOT iterating, so a filter context has nothing to do with iterations: filtering some rows in the *Customer* table does not mean iterating those rows. Furthermore, row context is not involved in measures natively. We shall be able to create a row context in a measure through **iterators**

Coffee Break

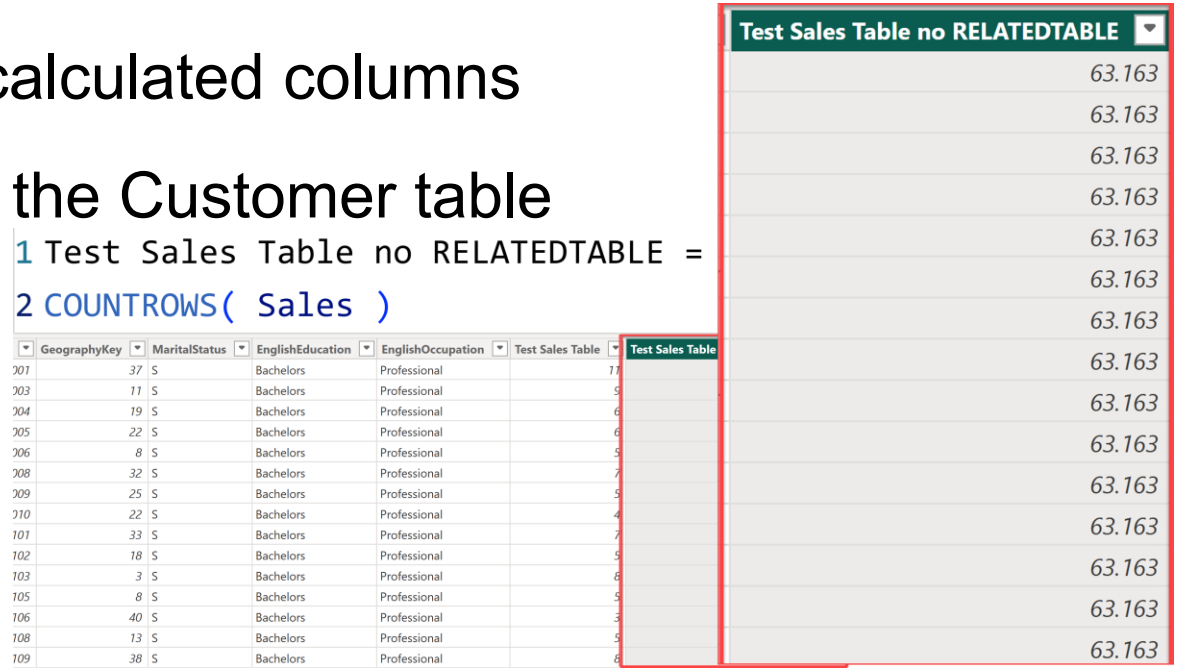
Iterators

Important points to learn in this section:

- Iterations do not filter and filters do not iterate
- SUM, MIN, MAX, AVERAGE, CONCATENATE… They do not exist in DAX ☺
- Understanding better the result of these calculated columns
- On the Sales table



On the Customer table



Iterators

Product	Price	Qty
A	10	2
B	5	5
C	1	1
A	10	10

What about these measures?

```
Revenue Measure = Sales[Qty] * Sales[Unit sale price]
```

A single value for column 'Qty' in table 'Sales' cannot be determined. This can happen when a measure or function formula refers to a column that contains many values without specifying an aggregation such as min, max, count, or sum to get a single result.

```
Revenue Measure = SUM(Sales[Qty]) * SUM(Sales[Unit sale price])
```

Product	Qty	Unit Price
A	1	10
B	10	5
C	5	10
Total	16	25

❌ 16 * 25 = 400

Product	Qty	Unit Price	
A	1	10	1 * 10
B	10	5	10 * 5
C	5	10	5 * 10
			10 + 50 + 50 = 110 row-by-row calculation

✅ 110 row-by row

What about these measures? Tipical answers

```
Revenue Measure = Sales[Qty] * Sales[Unit sale price]
```

A single value for column 'Qty' in table 'Sales' cannot be determined. This can happen when a measure or function formula refers to a column that contains many values without specifying an aggregation such as min, max, count, or sum to get a single result.

Answer: you need to create a calculated column with the above code of course it does not work in a measure!

```
Revenue Measure = SUM(Sales[Qty]) * SUM(Sales[Unit sale price])
```

Product	Qty	Unit Price
A	1	10
B	10	5
C	5	10
Total	16	25

❌ 16 * 25 = 400

Product	Qty	Unit Price	
A	1	10	1 * 10
B	10	5	10 * 5
C	5	10	5 * 10
			10 + 50 + 50 = 110 row-by-row calculation

✅ 110 row-by row

Answer: you need to create a calculated column with the code on top of the page and aggregate it with a measure!

What about these measures? Tipical answers

You need to create a calculated column with the code on top of the page and aggregate it with a measure!

Uhm… so I always need to create a calculated column when I need to do a row by row calculation?

No we do NOT! Iterators are functions that can create a row context on demand.

We can use them in measures, to create a row context on a table on which we need to do a row-by-row calculation and then aggregate the results, all within a measure!

What about these measures? Tipical answers

DAX rookie 1: You need to create a calculated column with the code on top of the page and aggregate it with a measure!

DAX rookie 2: Uhm… so I always need to create a calculated column when I need to do a row by row calculation?

DAX teacher: No you do NOT! Iterators are functions that can create a row context on demand. We can use them in measures, to create a row context on a table on which we need to do a row-by-row calculation and then aggregate the results, all within a measure!

What about these measures? Tipical answers

DAX rookie 1: Big deal! But I can go ahead with my create a column and then a measure-on-top-of-that thing

DAX teacher: This is a free world, so go ahead as you prefer, but I would avoid creating columns unless you use them for what they were originally created as a tool for BI: grouping, filtering, creating relationships

DAX rookie 1: But you also mentioned “to support a slow measure”

DAX teacher: uhm and where would the slow measure be, that you need to support?

When to avoid calculated columns

Why is the DAX teacher insisting on avoiding a calculated column in this case?

Because that column is not doing what calculated columns should do.

That column is created either because “that’s the way we do it in Excel” or because no full DAX knowledge is there

Calculated columns are great tools for their intended purpose, not for the purpose at hand

In fact calculated columns have a cost in terms of RAM (memory) and building them takes time, too: you need a measure anyway, so let’s do everything in measures, when a calculated column is not strictly needed.

Matching this idea with the power of iterators is of paramount importance

When to avoid calculated columns

The risk is to fill your RAM with useless calculated columns and not having room for useful ones!

So, in this case a measure is the solution

But… We said no iteration is available on measures by default

So how should I proceed?

Answer: Start and iteration of the Sales table, on every row calculate the product between unit price and quantity and then sum the results calculated in every row

What really stands behind the non existing SUM…

```
1 Sales Value SUMX =
2 SUMX (
3     Sales,
4     Sales[UnitPrice] * Sales[OrderQuantity]
5 )
```

Color	Nr Sales Rows	Sales Value	Sales Value SUMX
Black	10.212	€ 9.469.519	€ 9.469.519
Blue	4.144	€ 2.570.766	€ 2.570.766
Multi	4.115	€ 112.024	€ 112.024
NA	30.396	€ 457.964	€ 457.964
Red	5.029	€ 7.727.130	€ 7.727.130
Silver	3.610	€ 5.820.151	€ 5.820.151
White	593	€ 5.331	€ 5.331
Yellow	5.064	€ 5.806.442	€ 5.806.442
Total	**63.163**	**€ 31.969.327**	**€ 31.969.327**

What really stands behind the non existing SUM…

```
1 Sales Value SUMX =
2 SUMX (
3     Sales,
4     Sales[UnitPrice] * Sales[OrderQuantity]
5 )
```

```
1 Sales Value =
2 SUM ( Sales[SalesAmount] )
```

Questions to answer:
1 – the red part, how can it work, since the above is a measure and we have no row context in a measure?
2 – am I not creating anyway a calculated column in the above code?
3 – I do have a Sales[SalesAmount] column in the dataset, so why should I bother?
4 – I can create that column in case I do not have it (Power Query or DAX), so again why should I bother?
5 – you say SUM does not exist, so what about the *Sales Value* measure?

Iterator Functions

- SUMX
- MINX
- MAXX
- AVERAGEX
- CONCATENATEX

- An ‘iterator’ is a function that evaluates an expression for every row of a table and then aggerates the result
- Calculated Columns are “born” with row-by-row concept
- In measures, we have to create it with iterator functions.

```
Revenue ACT SUMX =
SUMX (
    'Sales',
    'Sales'[Qty] * 'Sales'[Unit sale price]
)
```

Product Name	Year, Month No	Country Name
Apple Juice	2022 (Year) + 6 (Month No)	Australia

Product Key	Date	Retailer Type	Employee Name	Country Code	Qty	Unit cost	Unit sale price
58	5. juni 2022	Discount Store	Matthew Goodrem	AUS	320	1,10	4,50
58	14. juni 2022	Restaurant	Scarlett Walsh	AUS	356	2,84	9,88
58	27. juni 2022	Specialty Store	Annabel Brown	AUS	393	13,42	64,55
58	21. juni 2022	Convenience Store	Edward Bell	AUS	399	1,58	5,64
58	15. juni 2022	Department Store	Madeleine Beck	AUS	437	1,87	2,88
58	1. juni 2022	Specialty Store	Justin Kidman	AUS	1.336	6,77	32,56
58	16. juni 2022	Convenience Store	Matthew Goodrem	AUS	1.457	8,06	33,13

Iterations row by row

320 * 4,50 = 1.439,68
356 * 9,88 = 3.518,42
393 * 64,55 = 25.368,23
399 * 5,64 = 2.250,60
437 * 2,88 = 1.258,47
1.336 * 32,56 = 43.505,10
1.457 * 33,13 = 48.265,46

Question:
What would MINX Return?
1.258,47

Question:
How many iterations were there?
7

Iterator (Scalar) Functions

- SUMX (Table, Expression)
- MINX (Table, Expression)
- MAXX(Table, Expression)
- AVERAGEX (Table, Expression)
- So, MIN, MAX, AVERAGE do not exist either!

- All the above do the same operations, only difference is the aggregation used

What real power Iterators give you

```
1 Sales Value SUMX =
2 SUMX (
3     Sales,
4     Sales[UnitPrice] * Sales[OrderQuantity]
5 )
```

```
1 Sales Value =
2 SUM ( Sales[SalesAmount] )
```

Not the red part, the green part: the **Table** input

```
1 Sales Value SUMX =
2 SUMX (
3     Sales,
4     Sales[UnitPrice] * Sales[OrderQuantity]
5 )
```

What real power Iterators give you

```
1 Sales Value SUMX =
2 SUMX (
3     Sales,
4     Sales[UnitPrice] * Sales[OrderQuantity]
5 )
```

```
1 Sales Value =
2 SUM ( Sales[SalesAmount] )
```

Of course, also the red part is important, it can be any DAX expression valid in the row context created, but the green part is what really brings you to the next level

```
1 Sales Value SUMX =
2 SUMX (
3     Sales,
4     Sales[UnitPrice] * Sales[OrderQuantity]
5 )
```

And that is because DAX has a LOT of **Table functions, that create virtual tables on the fly, on which you can iterate and perform calculations at the right granularity for example**

What real power Iterators give you

Let's see an example, a very simple calculation, in theory.

You want to restrict the calculation of the revenues to those amounts larger than a threshold.

You write this simple DAX

And it breaks at the total

Total should read *All Colors*

```
1 Sales Value threshold =
2 IF (
3     [Sales Value SUMX] > 500000,
4     [Sales Value SUMX]
5 )
```

Color	Nr Sales Rows	Sales Value SUMX	Sales Value threshold
Black	10.212	€ 9.469.519	€ 9.469.519
Blue	4.144	€ 2.570.766	€ 2.570.766
Multi	4.115	€ 112.024	
NA	30.396	€ 457.964	
Red	5.029	€ 7.727.130	€ 7.727.130
Silver	3.610	€ 5.820.151	€ 5.820.151
White	593	€ 5.331	
Yellow	5.064	€ 5.806.442	€ 5.806.442
Total	**63.163**	**€ 31.969.327**	**€ 31.969.327**

What real power Iterators give you

So the problem is simple:

We need to go always one color at a time
In order to filter out the low Sales

When we are on a single color (matrix rows), all is fine. But at the total I do not Consider the color at all

BUT

I can decide to go always color by color, I Only need a function that generates a table of all the unique colors

```
1 Sales Value threshold =
2 IF (
3     [Sales Value SUMX] > 500000,
4     [Sales Value SUMX]
5 )
```

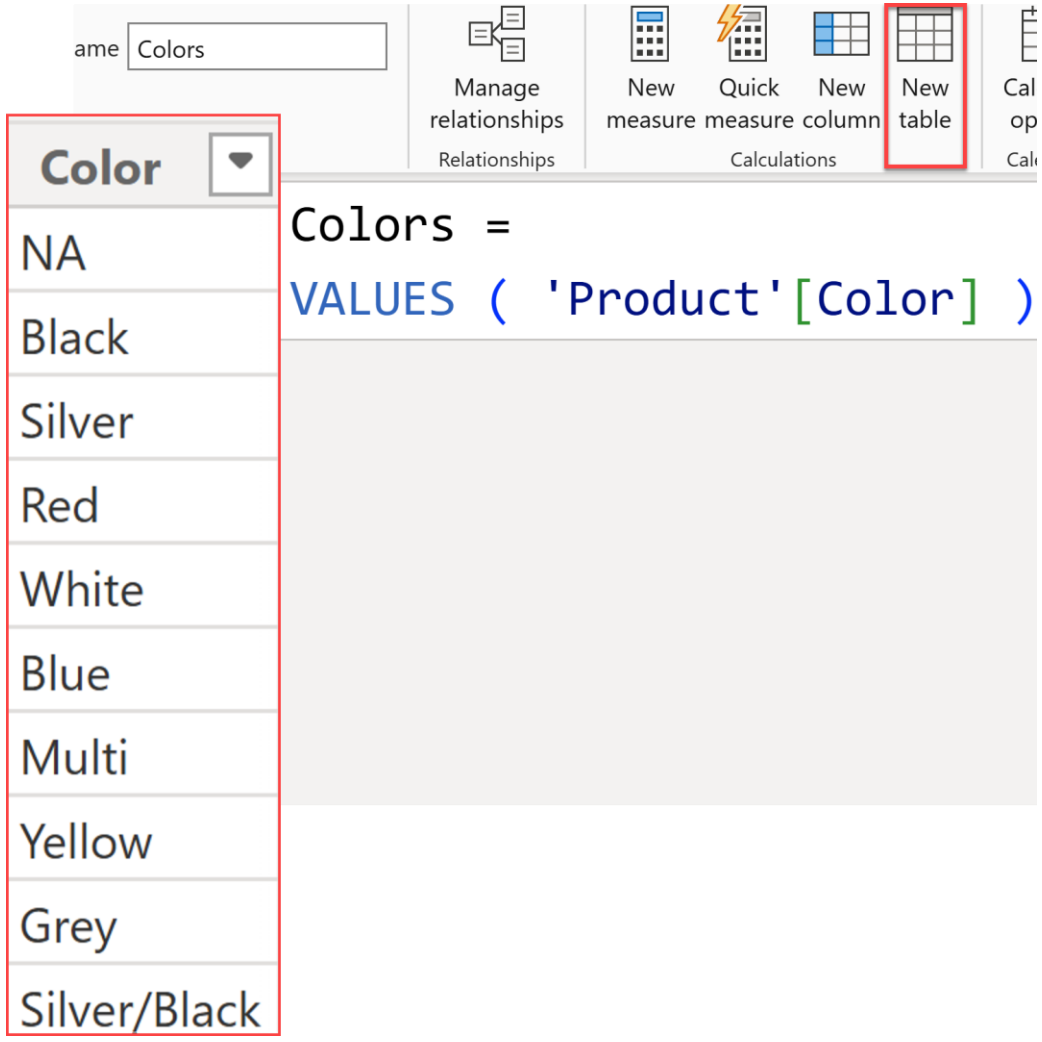
Color	Nr Sales Rows	Sales Value SUMX	Sales Value threshold
Black	10.212	€ 9.469.519	€ 9.469.519
Blue	4.144	€ 2.570.766	€ 2.570.766
Multi	4.115	€ 112.024	
NA	30.396	€ 457.964	
Red	5.029	€ 7.727.130	€ 7.727.130
Silver	3.610	€ 5.820.151	€ 5.820.151
White	593	€ 5.331	
Yellow	5.064	€ 5.806.442	€ 5.806.442
Total	**63.163**	**€ 31.969.327**	**€ 31.969.327**

What real power Iterators give you

I can decide to go always color by color, I Only need a function that generates a table of all the unique colors

VALUES (Table[Column])

Creates a table of all the unique values Of the input column.

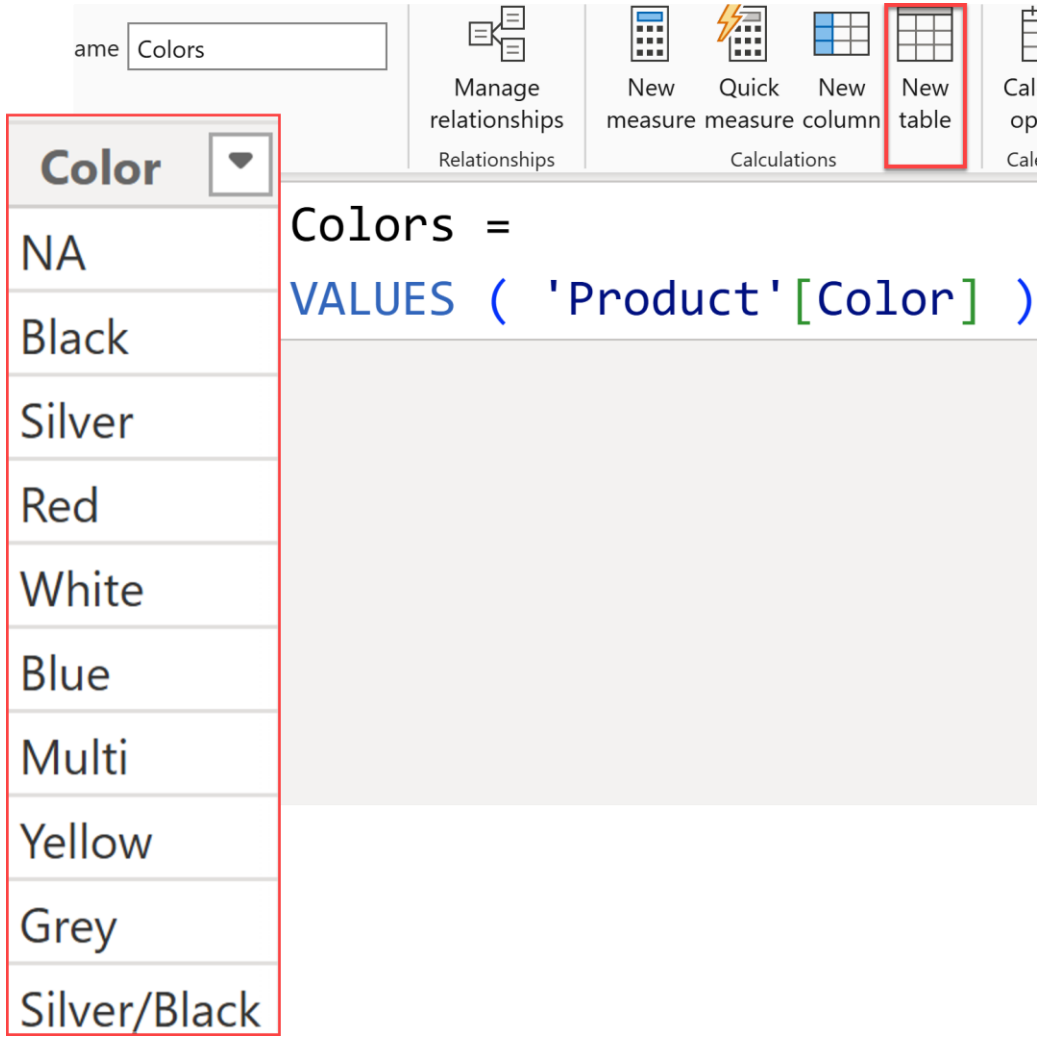


What real power Iterators give you

I need to create a row context on this table and check revenues vs the threshold

On each matrix row, only one color will Be visible (the filter context filters one row Of this table), but at the total I see all the Colors of the table and the calculation will Happen one row at a time

At the end I want to SUM the results



What real power Iterators give you

```
1 Sales Value threshold =
2 SUMX (
3     VALUES( 'Product'[Color] ),
4     IF (
5         [Sales Value SUMX] > 500000,
6         [Sales Value SUMX]
7     )
8 )
```

€ 31.969.327 **€ 31.394.008**

Color	Nr Sales Rows	Sales Value SUMX	Sales Value threshold
Black	10.212	€ 9.469.519	€ 9.469.519
Blue	4.144	€ 2.570.766	€ 2.570.766
Multi	4.115	€ 112.024	
NA	30.396	€ 457.964	
Red	5.029	€ 7.727.130	€ 7.727.130
Silver	3.610	€ 5.820.151	€ 5.820.151
White	593	€ 5.331	
Yellow	5.064	€ 5.806.442	€ 5.806.442
Total	**63.163**	**€ 31.969.327**	**€ 31.394.008**

Color	Sales Value SUMX
Black	*€ 9.469.519*
Blue	*€ 2.570.766*
Grey	
Multi	*€ 112.024*
NA	*€ 457.964*
Red	*€ 7.727.130*
Silver	*€ 5.820.151*
Silver/Black	
White	*€ 5.331*
Yellow	*€ 5.806.442*

Go row by row on the right table. This is mastering granularity

What real power Iterators give you

```
1 Sales Value threshold =
2 SUMX (
3     VALUES( 'Product'[Color] ),
4     IF (
5         [Sales Value SUMX] > 500000,
6         [Sales Value SUMX]
7     )
8 )
```

€ 31.969.327 € 31.394.008

Color	Nr Sales Rows	Sales Value SUMX	Sales Value threshold
Black	10.212	€ 9.469.519	€ 9.469.519
Blue	4.144	€ 2.570.766	€ 2.570.766
Multi	4.115	€ 112.024	
NA	30.396	€ 457.964	
Red	5.029	€ 7.727.130	€ 7.727.130
Silver	3.610	€ 5.820.151	€ 5.820.151
White	593	€ 5.331	
Yellow	5.064	€ 5.806.442	€ 5.806.442
Total	**63.163**	**€ 31.969.327**	**€ 31.394.008**

Color	Sales Value SUMX
Black	€ 9.469.519
Blue	€ 2.570.766
Grey	
Multi	€ 112.024
NA	€ 457.964
Red	€ 7.727.130
Silver	€ 5.820.151
Silver/Black	
White	€ 5.331
Yellow	€ 5.806.442

Filer context and row context now have a clear role: the filter context determines the rows that the row context can iterate. This is DAX

What real power Iterators give you

And now I do not want to go one product at a time anymore, can I go one let's say Education at a time? Yes I can but look a the total (of course unchanged): it does not show additivity anymore. We need still to define the granularity, let's say the

TUPLE

Color, Customer

let's reduce threshold to 5k

```
1 Sales Value threshold =
2 SUMX (
3     VALUES( 'Product'[Color] ),
4     IF (
5         [Sales Value SUMX] > 500000,
6         [Sales Value SUMX]
7     )
8 )
```

EnglishEducation	Nr Sales Rows	Sales Value SUMX	Sales Value threshold
Bachelors	18.869	€ 10.764.612	€ 10.598.117
Graduate Degree	11.074	€ 5.940.820	€ 5.836.250
High School	10.866	€ 5.065.723	€ 4.557.923
Partial College	17.419	€ 8.446.481	€ 8.289.348
Partial High School	4.935	€ 1.751.691	€ 508.991
Total	**63.163**	**€ 31.969.327**	**€ 31.394.008**

What real power Iterators give you

```
1 Sales Value threshold =
2 SUMX (
3     SUMMARIZE( Sales, 'Product'[Color], Customer[CustomerKey] ),
4     IF (
5         [Sales Value SUMX] > 5000,
6         [Sales Value SUMX]
7     )
8 )
```

EnglishEducation	Nr Sales Rows	Sales Value SUMX	Sales Value threshold
Bachelors	18.869	€ 10.764.612	€ 1.579.066
Partial College	17.419	€ 8.446.481	€ 1.233.789
Graduate Degree	11.074	€ 5.940.820	€ 836.303
High School	10.866	€ 5.065.723	€ 785.102
Partial High School	4.935	€ 1.751.691	€ 184.070
Total	**63.163**	**€ 31.969.327**	**€ 4.618.330**

Color	CustomerKey
Black	*29483*
Black	*29482*
Black	*29481*
Multi	*29480*
Yellow	*29480*
Black	*29480*
NA	*29480*
Black	*29479*
Blue	*29478*
NA	*29478*
Multi	*29477*
Blue	*29477*
Silver	*29476*
Silver	*29475*

What real power Iterators give you

```
1 Sales Value threshold =
2 SUMX (
3     SUMMARIZE( Sales, 'Product'[Color], Customer[CustomerKey] ),
4     IF (
5         [Sales Value SUMX] > 5000,
6         [Sales Value SUMX]
7     )
8 )
```

Color	Nr Sales Rows	Sales Value SUMX	Sales Value threshold
Yellow	5.064	€ 5.806.442	€ 1.328.753
Silver	3.610	€ 5.820.151	€ 1.309.971
Black	10.212	€ 9.469.519	€ 1.259.050
Blue	4.144	€ 2.570.766	€ 389.122
Red	5.029	€ 7.727.130	€ 331.434
Multi	4.115	€ 112.024	
NA	30.396	€ 457.964	
White	593	€ 5.331	
Total	**63.163**	**€ 31.969.327**	**€ 4.618.330**

Back to the beginning of the section

Important points to learn in this section:

- Understanding better the result of these calculated columns
- On the Sales table

```
1 Another Calculated Column =
2 COUNTROWS( Sales )
```

CustomerKey	SalesTerritoryKey	OrderQuantity	UnitPrice	SalesAmount	My Calcu
11082	4	1	$4,99	$5	standard
11088	4	1	$4,99	$5	standard
11135	4	1	$4,99	$5	standard
11161	4	1	$4,99	$5	standard
11171	4	1	$4,99	$5	standard
11175	4	1	$4,99	$5	standard
11208	4	1	$4,99	$5	standard
11227	4	1	$4,99	$5	standard
11252	4	1	$4,99	$5	standard
11267	4	1	$4,99	$5	standard
11273	4	1	$4,99	$5	standard
11281	4	1	$4,99	$5	standard

Another Calculated Column
63.163
63.163
63.163
63.163
63.163
63.163
63.163
63.163
63.163
63.163
63.163

- On the Customer table

```
1 Test Sales Table no RELATEDTABLE =
2 COUNTROWS( Sales )
```

GeographyKey	MaritalStatus	EnglishEducation	EnglishOccupation	Test Sales Table	Test Sales Table
37	S	Bachelors	Professional	11	
11	S	Bachelors	Professional	9	
19	S	Bachelors	Professional	6	
22	S	Bachelors	Professional	6	
8	S	Bachelors	Professional	5	
32	S	Bachelors	Professional	7	
25	S	Bachelors	Professional	5	
22	S	Bachelors	Professional	4	
33	S	Bachelors	Professional	7	
18	S	Bachelors	Professional	5	
3	S	Bachelors	Professional	6	
8	S	Bachelors	Professional	5	
40	S	Bachelors	Professional	3	
13	S	Bachelors	Professional	5	
38	S	Bachelors	Professional	8	

Test Sales Table no RELATEDTABLE
63.163
63.163
63.163
63.163
63.163
63.163
63.163
63.163
63.163
63.163
63.163
63.163
63.163
63.163
63.163

Morning Summary and Q&A

Lunch Break

Table functions

Important points to learn in this section:

- FILTER does not filter ☺
- Variables in DAX are constant values/tables ☺
- Table Functions can be iterators too

Table Functions in Iterators

Table functions to start with

FILTER (Table/TableExpression, FilterExpression)
VALUES (Table[Column])
DISTINCT (Table[Column])
ALL (Table)
SUMMARIZE (Table, Column1, Column2, …)
ADDCOLUMNS (Table, ColumnName1, ColumnExpression1, …)
SUMMARIZECOLUMNS ()

Let’s check daxguide for each of them

Table Functions in Iterators

Table functions to start with

FILTER (Table/TableExpression, FilterExpression)
VALUES (Table[Column])
DISTINCT (Table[Column])
ALL (Table)
SUMMARIZE (Table, Column1, Column2, …)
ADDCOLUMNS (Table, ColumnName1, ColumnExpression1, …)

Let’s see them in action creating calculated tables with each

Table Functions in Iterators

FILTER (Table/TableExpression, FilterExpression)

FILTER iterates the input table and marks each row based on the result of the Expression (TRUE or FALSE). Those rows where the Expression returns TRUE are included in the output table

So FILTER is an Iterator like SUMX, MINX, but FILTER is NOT a scalar iterator, it is a table function that is also an iterator

So a measure/calculated column cannot have an expression that starts with FILTER as a measure/calculated column code must create a scalar value

A calculated table or a virtual table, on the contrary, can have a DAX code starting with a call to FILTER

Table Functions in Iterators

FILTER (Table/TableExpression, FilterExpression)

```
1 Sales to Bachelors =
2 FILTER (
3     Sales,
4     RELATED ( Customer[EnglishEducation] ) = "Bachelors"
5 )
```

OrderDate	ShipDateKey	CustomerKey	SalesTerritoryKey	OrderQuantity	UnitPrice	SalesAmount	My Calculated Column
'7 12/06/2004 00:00:00	20040619	11001	9	1	4,99	4,99	standard qty
'7 20/07/2003 00:00:00	20030727	11001	9	1	4,99	4,99	standard qty
'7 09/07/2003 00:00:00	20030716	11003	9	1	4,99	4,99	standard qty
'7 02/07/2003 00:00:00	20030709	11006	9	1	4,99	4,99	standard qty
'7 07/07/2003 00:00:00	20030714	11008	9	1	4,99	4,99	standard qty
'7 02/01/2004 00:00:00	20040109	11099	9	1	4,99	4,99	standard qty
'7 07/09/2003 00:00:00	20030914	11099	9	1	4,99	4,99	standard qty
'7 31/12/2003 00:00:00	20040107	11101	9	1	4,99	4,99	standard qty
'7 11/06/2004 00:00:00	20040618	11102	9	1	4,99	4,99	standard qty

This is a calculated table (18.869 rows) that, as a calculated column, is not influenced by filters (and likely we can inject filters – later – through CALCULATE)

Table Functions in Iterators

```
1 Sales Value to Bachelors =
2 SUMX (
3     FILTER (
4         Sales,
5         RELATED ( Customer[EnglishEducation] ) = "Bachelors"
6     ),
7     Sales[UnitPrice] * Sales[OrderQuantity]
8 )
```

Color	Nr Sales Rows	Sales Value SUMX	Sales Value to Bachelors
Black	10.212	€ 9.469.519	€ 3.308.032
Blue	4.144	€ 2.570.766	€ 853.361
Multi	4.115	€ 112.024	€ 32.922
NA	30.396	€ 457.964	€ 132.108
Red	5.029	€ 7.727.130	€ 2.404.822
Silver	3.610	€ 5.820.151	€ 1.914.169
White	593	€ 5.331	€ 1.465
Yellow	5.064	€ 5.806.442	€ 2.117.733
Total	**63.163**	**€ 31.969.327**	**€ 10.764.612**

FILTER (
    Table/TableExpression,
    FilterExpression
)

What if we use this table expression in a measure?

Table Functions in Iterators

```
1 Sales Value to Bachelors =
2 SUMX (
3     FILTER (
4         Sales,
5         RELATED ( Customer[EnglishEducation] ) = "Bachelors"
6     ),
7     Sales[UnitPrice] * Sales[OrderQuantity]
8 )
```

As a calculated table we had 18.869 rows

Color	Nr Sales Rows	Sales Value SUMX	Sales Value to Bachelors
Black	10.212	€ 9.469.519	€ 3.308.032
Blue	4.144	€ 2.570.766	€ 853.361
Multi	4.115	€ 112.024	€ 32.922
NA	30.396	€ 457.964	€ 132.108
Red	5.029	€ 7.727.130	€ 2.404.822
Silver	3.610	€ 5.820.151	€ 1.914.169
White	593	€ 5.331	€ 1.465
Yellow	5.064	€ 5.806.442	€ 2.117.733
Total	**63.163**	**€ 31.969.327**	**€ 10.764.612**

Table Functions in Iterators

```
1 Nr Sales Rows to Bachelors =
2 COUNTROWS (
3     FILTER (
4         Sales,
5         RELATED ( Customer[EnglishEducation] ) = "Bachelors"
6     )
7 )
```

Color	Nr Sales Rows	Sales Value SUMX	Nr Sales Rows to Bachelors	Sales Value to Bachelors
Black	10.212	€ 9.469.519	3.279	€ 3.308.032
Blue	4.144	€ 2.570.766	1.238	€ 853.361
Multi	4.115	€ 112.024	1.213	€ 32.922
NA	30.396	€ 457.964	8.527	€ 132.108
Red	5.029	€ 7.727.130	1.520	€ 2.404.822
Silver	3.610	€ 5.820.151	1.160	€ 1.914.169
White	593	€ 5.331	163	€ 1.465
Yellow	5.064	€ 5.806.442	1.769	€ 2.117.733
Total	**63.163**	**€ 31.969.327**	**18.869**	**€ 10.764.612**

As a calculated table we had 18.869 rows,
As a virtual table in the measure, we see this value only at the total where there is no filter on the color

On a single color, the virtual table is filtered by the filter context!

Again: filter context and row context together

Table Functions in Iterators

```
1 Sales Value to Bachelors PCT =
2 DIVIDE(
3     [Sales Value to Bachelors],
4     [Sales Value]
5 )
```

Color	Sales Value SUMX	Sales Value to Bachelors	Sales Value to Bachelors PCT
Black	€ 9.469.519	€ 3.308.032	34,93%
Blue	€ 2.570.766	€ 853.361	33,19%
Multi	€ 112.024	€ 32.922	29,39%
NA	€ 457.964	€ 132.108	28,85%
Red	€ 7.727.130	€ 2.404.822	31,12%
Silver	€ 5.820.151	€ 1.914.169	32,89%
White	€ 5.331	€ 1.465	27,49%
Yellow	€ 5.806.442	€ 2.117.733	36,47%
Total	**€ 31.969.327**	**€ 10.764.612**	**33,67%**

What is the purpose of the *Sales Value to Bachelors* measure at the end?

```
1 Sales Value to Bachelors =
2 SUMX (
3     FILTER (
4         Sales,
5         RELATED ( Customer[EnglishEducation] ) = "Bachelors"
6     ),
7     Sales[UnitPrice] * Sales[OrderQuantity]
8 )
```

Table Functions in Iterators

Break/Discussion:

1) Is it clear how much power we are gaining through table functions?

2) Do we also notice that the DAX is becoming long and pretty complex? (CALCULATE/CALCULATETABLE will help on this – later)

Considering 2) then why are we doing all this and did not run to CALCULATE/CALCULATETABLE directly?

Important answer: because filter ARE tables in DAX and in order to use well CALCULATE/CALCULATETABLE we need to be able to create filters through tables, so what we are doing now is VERY important in perspective

Table Functions in Iterators

Break/Discussion:

3) Why did I insist that FILTER does not filter?

Table Functions in Iterators

Break/Discussion:

3) Why did I insist that FILTER does not filter?

Variables in DAX

They are constant values

They are calculated only once (at creation) and never again

They optimize and simplify the code and make it more readable, yet longer

They can be defined in any point of a DAX expression

They can result in a table or a scalar value

Perfect for the above case!

Create the two measures and insert into a visual
Do we get the expected result?

```
Region Average =
AVERAGEX(
        VALUES('Sales Territory'[Sales Territory Region]),
        [Sales]
)
```

```
Regions Buying More Than Average =
COUNTROWS(
    FILTER(
        VALUES('Sales Territory'[Sales Territory Region]),
        [Sales] > [Region Average] )
)
```

VAR
Variables are constant

- Performance
- Readability

- Naming with _
- Always have a _Result for easier debugging

```
Regions Buying More Than Average =
VAR _Avg =
AVERAGEX(
        VALUES('Sales Territory'[Sales Territory Region]),
        [Sales]
)

VAR _Result =
COUNTROWS(
    FILTER(
        VALUES('Sales Territory'[Sales Territory Region]),
        [Sales] > _Avg
    )
)

RETURN
_Result
```

Context Transition

- There is no filter context in a row context. Unless you specify a context transition.
- Context transition is the process of converting a row context to an equivalent filter context
- Use CALCULATE or a Measure Reference in a row-context to enable context transition.
- Every measure reference always has an implicit CALCULATE surrounding it
- Context transition is expensive operation. If used during an iteration on a table with 10 columns and one million rows, then CALCULATE needs to apply 10 filters, one million times.
- Context transition does not only filter one row
- Context transition uses columns that are not present in the formular
- Calculated tables and columns aren't evaluated within filter context

Context Transition, CALCULATE and CALCULATETABLE

What you learn in this section

- What context transition is, when it happens and how it happens
- How to control it
- What CALCULATE/CALCULATETABLE can do, further than context transition

- Loose end:
    1. RELATEDTABLE / CALCULATETABLE
    2. Injection of filters through CALCULATE/CALCULATETABLE
    3. The variation 2 os Sales Value Customer Trechold puzzle

Context Transition – a step back for a moment

- Consider now the following DAX expression in a calculated column in the *Customer* table, what do you expect as a result?

```
1 Test Sales Table no RELATEDTABLE =
2 COUNTROWS( Sales )
```

	GeographyKey	MaritalStatus	EnglishEducation	EnglishOccupation	Test Sales Table	Test Sales Table no RELATEDTABLE
001	37	S	Bachelors	Professional	11	63.163
003	11	S	Bachelors	Professional	9	63.163
004	19	S	Bachelors	Professional	6	63.163
005	22	S	Bachelors	Professional	6	63.163
006	8	S	Bachelors	Professional	5	63.163

Context Transition – a step back for a moment

- Let's wrap the code inside a CALCULATE call

```
1 Test Sales Table no RELATEDTABLE =
2 CALCULATE(
3     COUNTROWS( Sales )
4 )
```

	GeographyKey	MaritalStatus	EnglishEducation	EnglishOccupation	Test Sales Table	Test Sales Table no RELATEDTABLE
001	37	S	Bachelors	Professional	11	11
003	11	S	Bachelors	Professional	9	9
004	19	S	Bachelors	Professional	6	6
005	22	S	Bachelors	Professional	6	6
006	8	S	Bachelors	Professional	5	5
008	32	S	Bachelors	Professional	7	7
009	25	S	Bachelors	Professional	5	5

Context Transition – a step back for a moment

- Let's wrap the code inside a CALCULATE call

```
1 Test Sales Table no RELATEDTABLE =
2 CALCULATE(
3     COUNTROWS( Sales )
4 )
```

	GeographyKey	MaritalStatus	EnglishEducation	EnglishOccupation	Test Sales Table	Test Sales Table no RELATEDTABLE
001	37	S	Bachelors	Professional	11	11
003	11	S	Bachelors	Professional	9	9
004	19	S	Bachelors	Professional	6	6
005	22	S	Bachelors	Professional	6	6
006	8	S	Bachelors	Professional	5	5
008	32	S	Bachelors	Professional	7	7
009	25	S	Bachelors	Professional	5	5

- What is happening? CALCULATE has triggered a context transition, turning the row context created automatically by the engine during the evaluation of the DAX code of the calculated column into an equivalent filter context

Context Transition – a step back for a moment

- Let’s see what is happening in detail

```
1 Test Sales Table no RELATEDTABLE =
2 CALCULATE(
3     COUNTROWS( Sales )
4 )
```

GeographyKey	MaritalStatus	EnglishEducation	EnglishOccupation	Test Sales Table	Test Sales Table no RELATEDTABLE
37	S	Bachelors	Professional	11	11
11	S	Bachelors	Professional	9	9
19	S	Bachelors	Professional	6	6
22	S	Bachelors	Professional	6	6
8	S	Bachelors	Professional	5	5
32	S	Bachelors	Professional	7	7
25	S	Bachelors	Professional	5	5

Context Transition – a step back for a moment

- Let’s see what is happening in detail

```
1 Test Sales Table no RELATEDTABLE =
2 CALCULATE(
3     COUNTROWS( Sales )
4 )
```

GeographyKey	MaritalStatus	EnglishEducation	EnglishOccupation	Test Sales Table	Test Sales Table no RELATEDTABLE
37	S	Bachelors	Professional	11	11
11	S	Bachelors	Professional	9	9
19	S	Bachelors	Professional	6	6
22	S	Bachelors	Professional	6	6
8	S	Bachelors	Professional	5	5
32	S	Bachelors	Professional	7	7
25	S	Bachelors	Professional	5	5

Context Transition – a step back for a moment

- Let's now substitute the call to a measure....

```
1 Test Sales Table no RELATEDTABLE =
2 [Nr Sales Rows]
```

	GeographyKey	MaritalStatus	EnglishEducation	EnglishOccupation	Test Sales Table	Test Sales Table no RELATEDTABLE
)01	37	S	Bachelors	Professional	11	11
)03	11	S	Bachelors	Professional	9	9
)04	19	S	Bachelors	Professional	6	6
)05	22	S	Bachelors	Professional	6	6
)06	8	S	Bachelors	Professional	5	5
)08	32	S	Bachelors	Professional	7	7
)09	25	S	Bachelors	Professional	5	5
)10	22	S	Bachelors	Professional	4	4
101	33	S	Bachelors	Professional	7	7

- What is happening now? There is no trace of any CALCULATE call in the measure DAX code that is called in the calculated column. BUT we observe still context transition, so... ????

Context Transition – a step back for a moment

- Let's now reveal the secret!
- Whenever you call a measure with its name (and not its explicit code), there is a hidden CALCULATE wrapped around it
- So this should explain:

```
1 Test Sales Table no RELATEDTABLE =
2 [Nr Sales Rows]
3 -- the above call to a measure is turned not into its code as it is
4 -- but with a wrapped CALCULATE around it so...
5 -- CALCULATE ( SUMX ( Sales, Sales[UnitPrice] * Sales[OrderQuantity] ) )
```

GeographyKey	MaritalStatus	EnglishEducation	EnglishOccupation	Test Sales Table	Test Sales Table no RELATEDTABLE
001	37	S	Bachelors	Professional	11	11
003	11	S	Bachelors	Professional	9	9
004	19	S	Bachelors	Professional	6	6
005	22	S	Bachelors	Professional	6	6
006	8	S	Bachelors	Professional	5	5
008	32	S	Bachelors	Professional	7	7
009	25	S	Bachelors	Professional	5	5
010	22	S	Bachelors	Professional	4	4
101	33	S	Bachelors	Professional	7	7
102	18	S	Bachelors	Professional	5	5
103	3	S	Bachelors	Professional	8	8
105	8	S	Bachelors	Professional	5	5
106	40	S	Bachelors	Professional	3	3
108	13	S	Bachelors	Professional	5	5
109	38	S	Bachelors	Professional	8	8

Context Transition

- Context transition is the process of converting a row context to an equivalent filter context
- Use CALCULATE or a Measure Reference in a row-context to enable context transition.
- Every measure reference always has an implicit CALCULATE surrounding it
- Context transition is expensive operation. If used during an iteration on a table with 10 columns and one million rows, then CALCULATE needs to apply 10 filters, one million times.
- Context transition does not only filter one row
- Context transition uses columns that are not present in the formular
- Calculated tables and columns aren't evaluated within filter context

Context Transition

- Do we always want it? No, we can discuss when ☺
- Can we avoid it happens? Yes
- In fact, CALCULATE does many other things, and one of the other things it does is to add or remove filters before evaluating a DAX expression
- Among the filters that can be removed, you find the context transition ones!

```
1 Test Sales Table no RELATEDTABLE =
2 CALCULATE(
3     [Nr Sales Rows],
4     REMOVEFILTERS( )
5 )
6 -- the above call to a measure is turned not into its code as it is
7 -- but with a wrapped CALCULATE around it so...
8 -- CALCULATE ( SUMX ( Sales, Sales[UnitPrice] * Sales[OrderQuantity] )
```

GeographyKey	MaritalStatus	EnglishEducation	EnglishOccupation	Test Sales Table	Test Sales Table no RELATEDTABLE
37	S	Bachelors	Professional	11	63.163
11	S	Bachelors	Professional	9	63.163
19	S	Bachelors	Professional	6	63.163
22	S	Bachelors	Professional	6	63.163
8	S	Bachelors	Professional	5	63.163
32	S	Bachelors	Professional	7	63.163

Context Transition

- When do not we want context transition? On tables without primary key and in general when it can be a performance issue

Last issue to solve: Filters Injection/Management – another application of CALCULATE

- Do you remember we noticed the DAX code becoming more and more complex?

- CALCULATE can help also here!

Last issue to solve: Filters Injection/Management – another application of CALCULATE

- Do you remember we noticed the DAX code becoming more and more complex?

- CALCULATE can help also here!

Back to Table Functions: Table Functions in Iterators

Break/Discussion:

1) Is it clear how much power we are gaining through table functions?

2) Do we also notice that the DAX is becoming long and pretty complex? (CALCULATE/CALCULATETABLE will help on this – later)

Considering 2) then why are we doing all this and did not run to CALCULATE/CALCULATETABLE directly?

Important answer: because filter ARE tables in DAX and in order to use well CALCULATE/CALCULATETABLE we need to be able to create filters through tables, so what we are doing now is VERY important in perspective

Back to Table Functions: Table Functions in Iterators

```
1 Sales Value to Bachelors =
2 SUMX (
3     FILTER (
4         Sales,
5         RELATED ( Customer[EnglishEducation] ) = "Bachelors"
6     ),
7     Sales[UnitPrice] * Sales[OrderQuantity]
8 )
```

Color	Sales Value SUMX	Sales Value to Bachelors
Black	€ 9.469.519	€ 3.308.032
Blue	€ 2.570.766	€ 853.361
Multi	€ 112.024	€ 32.922
NA	€ 457.964	€ 132.108
Red	€ 7.727.130	€ 2.404.822
Silver	€ 5.820.151	€ 1.914.169
White	€ 5.331	€ 1.465
Yellow	€ 5.806.442	€ 2.117.733
Totale	**€ 31.969.327**	**€ 10.764.612**

```
1 Sales Value =
2 SUM ( Sales[SalesAmount] )
```

This way of modifying measures has a lot of issues, let's list them

Back to Table Functions: Table Functions in Iterators

```
1 Sales Value to Bachelors CALCULATE =
2 CALCULATE(
3     [Sales Value],
4     Customer[EnglishEducation] = "Bachelors"
5 )
```

```
1 Sales Value =
2 SUM ( Sales[SalesAmount] )
```

This way of modifying measures solves all the issues

Color	Sales Value SUMX	Sales Value to Bachelors	Sales Value to Bachelors CALCULATE
Black	€ 9.469.519	€ 3.308.032	€ 3.308.032
Blue	€ 2.570.766	€ 853.361	€ 853.361
Multi	€ 112.024	€ 32.922	€ 32.922
NA	€ 457.964	€ 132.108	€ 132.108
Red	€ 7.727.130	€ 2.404.822	€ 2.404.822
Silver	€ 5.820.151	€ 1.914.169	€ 1.914.169
White	€ 5.331	€ 1.465	€ 1.465
Yellow	€ 5.806.442	€ 2.117.733	€ 2.117.733
Totale	**€ 31.969.327**	**€ 10.764.612**	**€ 10.764.612**

Back to Table Functions: Table Functions in Iterators

```
1 Sales Value to Bachelors CALCULATE =
2 CALCULATE(
3     [Sales Value],
4     Customer[EnglishEducation] = "Bachelors"
5 )
```

- This way of modifying measures solves all the issues

- Customer[EnglishEducation] = "Bachelors« is a table! And therefore a filter injected by CALCULATE

- Let’s see the real (table) expression executed when you write this apparent predicate

Back to Table Functions: Table Functions in Iterators

```
1 Sales Value to Bachelors CALCULATE =
2 CALCULATE(
3     [Sales Value],
4     Customer[EnglishEducation] = "Bachelors"
5 )
```

- Customer[EnglishEducation] = "Bachelors« is a table! And therefore a filter injected by CALCULATE

- Let’s see the real (table) expression executed when you write this apparent predicate

Color	Sales Value SUMX	Sales Value to Bachelors	Sales Value to Bachelors CALCULATE
Black	€ 9.469.519	€ 3.308.032	€ 3.308.032
Blue	€ 2.570.766	€ 853.361	€ 853.361
Multi	€ 112.024	€ 32.922	€ 32.922
NA	€ 457.964	€ 132.108	€ 132.108
Red	€ 7.727.130	€ 2.404.822	€ 2.404.822
Silver	€ 5.820.151	€ 1.914.169	€ 1.914.169
White	€ 5.331	€ 1.465	€ 1.465
Yellow	€ 5.806.442	€ 2.117.733	€ 2.117.733
Totale	**€ 31.969.327**	**€ 10.764.612**	**€ 10.764.612**

Coffee Break

Wrap up excersise

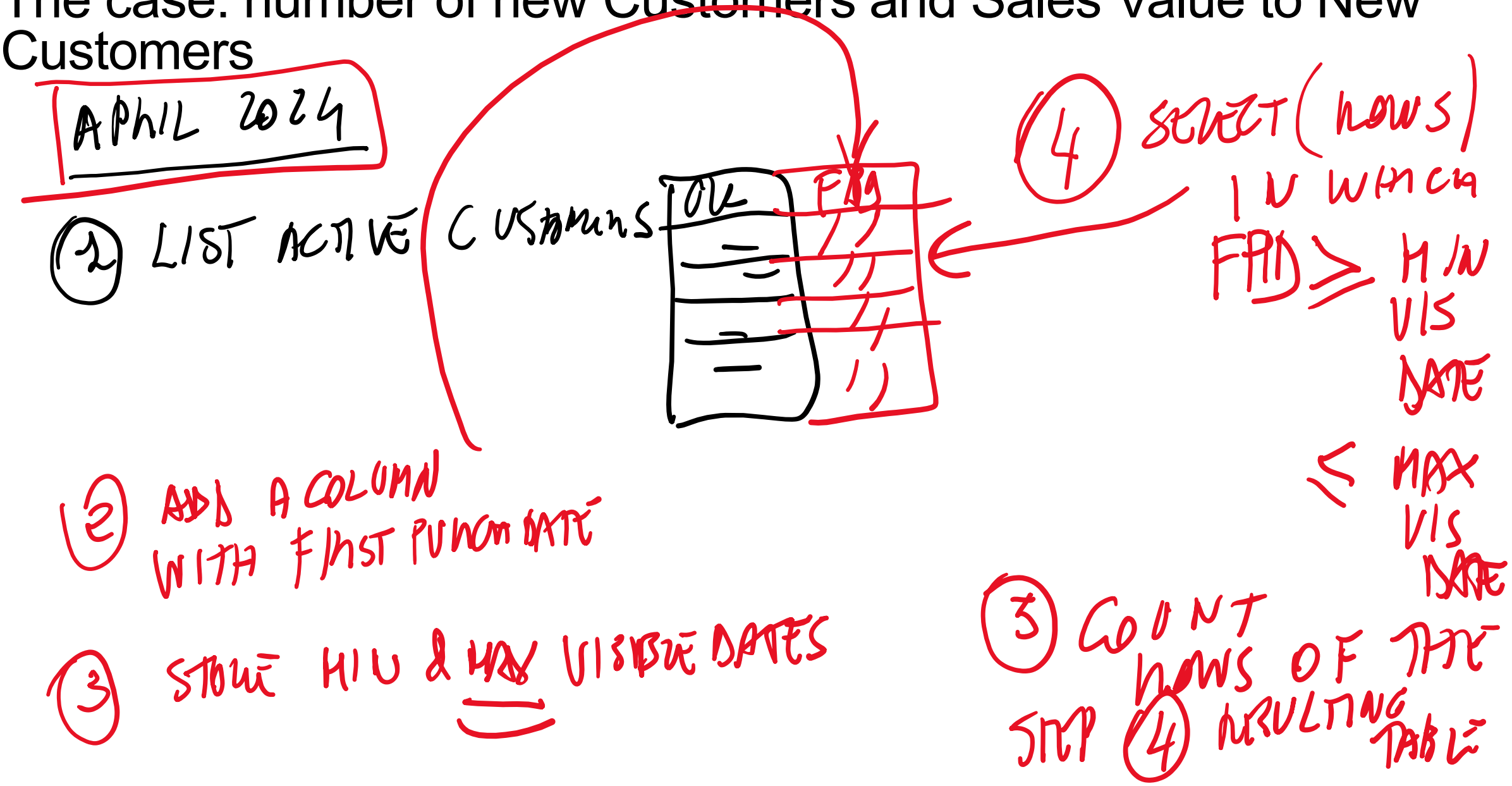
The case: Number of New Customers

we will see:

- the importance of definitions
- the importance of designing the solution with paper and pencil before writing DAX
- the interaction between context transition and REMOVEFILTERS in CALCULATE

A CUST IS NEW IN A PERIOD IF IT BOUGHT IN THAT PERIOD AND IT NEVER BOUGHT BEFORE THE BEGINNING OF THE PERIOD (IGNORING THE CURRENT SELECTION)

The case: number of new Customers and Sales Value to New Customers



The case: number of new Customers and Sales Value to New Customers

Expanded Tables

- When you ask for a table, you get the expanded table
- Looks like the One Big Table (OBT) as it joins the related table columns
- If you refer to the fact table of Sales, it will refer to all native columns in the Fact table + all related columns from the dimension tables that have a one-to-many relationship to the fact table
- For performance optimization, always focus on only including the needed columns. Example of bad practice:

```
SUMX (
    FILTER (
        'Fact Table',
         Expression
    )
)
```

- "Never filter tables, always filter columns"
  - SQLBI

Anything else on Expanded Tables?

- Yes, consider this measure and this visual

```
1 ALL Revenues Expanded Table =
2 CALCULATE(
3     [Revenues SUMX],
4     REMOVEFILTERS ( Sales )
5 )
```

EnglishEducation	Revenues SUMX	ALL Revenues Expanded Table
Bachelors	3.308.032	31.969.327
Graduate Degree	1.710.020	31.969.327
High School	1.433.138	31.969.327
Partial College	2.509.338	31.969.327
Partial High School	508.991	31.969.327
Total	**9.469.519**	**31.969.327**



How is it possible that removing a filter from a table that is NOT filtered (filters are only on Customer [EnglishEducation] and Product[Color], NOT on Sales columns), results is seeing all Revenues values?

Because whenever you refer to a table, again, you refer to the expanded table. Sales, expanded, includes all columns from Product, Customer, Calendar… ALL columns!

When you ask for a table, you will get the expanded table

Table: Sales (63.163 rows)

Product Key	Order Date	Ship Date Key	Customer Key	Sales Territory Key	Order Quantity	Unit Price	Sales Amount
477	2025-05-04	20250516	11082	4	1	$4,99	$5
477	2025-05-10	20250522	11088	4	1	$4,99	$5
477	2024-10-14	20241027	11135	4	1	$4,99	$5
477	2024-11-27	20241210	11161	4	1	$4,99	$5
477	2025-05-05	20250517	11171	4	1	$4,99	$5
477	2025-05-01	20250513	11175	4	1	$4,99	$5
477	2024-08-30	20240912	11208	4	1	$4,99	$5
477	2024-08-08	20240821	11227	4	1	$4,99	$5
477	2025-06-15	20250627	11252	4	1	$4,99	$5
477	2024-07-05	20240718	11267	4	1	$4,99	$5
477	2024-11-09	20241122	11273	4	1	$4,99	$5
477	2024-07-09	20240722	11281	4	1	$4,99	$5
477	2024-06-28	20240711	11292	4	1	$4,99	$5
477	2024-08-19	20240901	11298	4	1	$4,99	$5
477	2024-08-17	20240830	11528	4	1	$4,99	$5
477	2024-08-05	20240818	11683	4	1	$4,99	$5

When you ask for a table, you will get the **expanded** table

Table: Sales (63.163 rows)

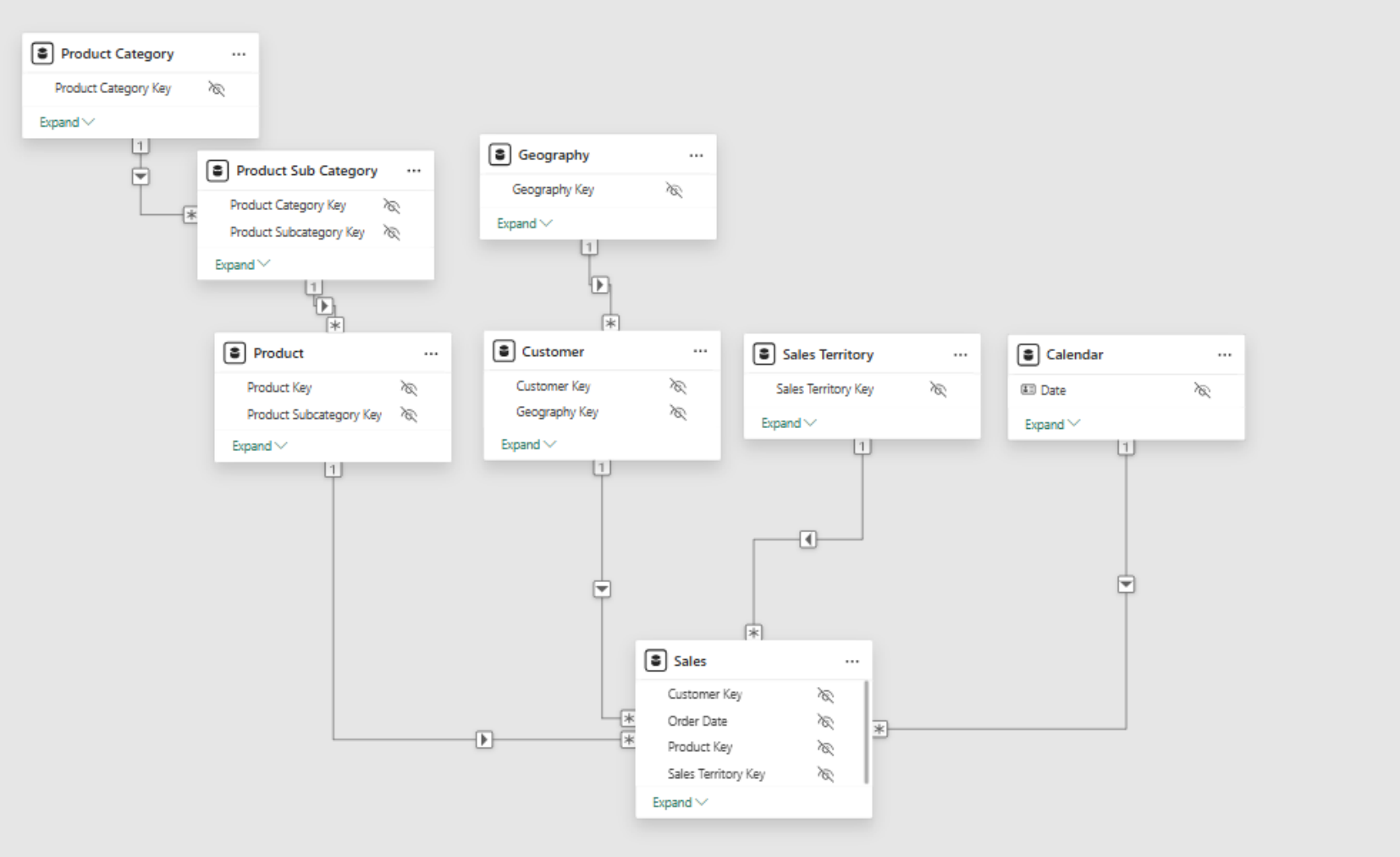
```
1 (R) Product Name = RELATED ( Product[English Product Name] )
```

Product Key	Order Date	Ship Date Key	Customer Key	Sales Territory Key	Order Quantity	Unit Price	Sales Amount	(R) Product Name	(R) English Product Subcategory Name	(R) English Product Category Name
477	2025-05-04	20250516	11082	4	1	$4,99	$5	Water Bottle - 30 oz.	Bottles and Cages	Accessories
477	2025-05-10	20250522	11088	4	1	$4,99	$5	Water Bottle - 30 oz.	Bottles and Cages	Accessories
477	2024-10-14	20241027	11135	4	1	$4,99	$5	Water Bottle - 30 oz.	Bottles and Cages	Accessories
477	2024-11-27	20241210	11161	4	1	$4,99	$5	Water Bottle - 30 oz.	Bottles and Cages	Accessories
477	2025-05-05	20250517	11171	4	1	$4,99	$5	Water Bottle - 30 oz.	Bottles and Cages	Accessories
477	2025-05-01	20250513	11175	4	1	$4,99	$5	Water Bottle - 30 oz.	Bottles and Cages	Accessories
477	2024-08-30	20240912	11208	4	1	$4,99	$5	Water Bottle - 30 oz.	Bottles and Cages	Accessories
477	2024-08-08	20240821	11227	4	1	$4,99	$5	Water Bottle - 30 oz.	Bottles and Cages	Accessories
477	2025-06-15	20250627	11252	4	1	$4,99	$5	Water Bottle - 30 oz.	Bottles and Cages	Accessories
477	2024-07-05	20240718	11267	4	1	$4,99	$5	Water Bottle - 30 oz.	Bottles and Cages	Accessories
477	2024-11-09	20241122	11273	4	1	$4,99	$5	Water Bottle - 30 oz.	Bottles and Cages	Accessories
477	2024-07-09	20240722	11281	4	1	$4,99	$5	Water Bottle - 30 oz.	Bottles and Cages	Accessories
477	2024-06-28	20240711	11292	4	1	$4,99	$5	Water Bottle - 30 oz.	Bottles and Cages	Accessories
477	2024-08-19	20240901	11298	4	1	$4,99	$5	Water Bottle - 30 oz.	Bottles and Cages	Accessories
477	2024-08-17	20240830	11528	4	1	$4,99	$5	Water Bottle - 30 oz.	Bottles and Cages	Accessories
477	2024-08-05	20240818	11683	4	1	$4,99	$5	Water Bottle - 30 oz.	Bottles and Cages	Accessories

Native Fact Columns

Related Column

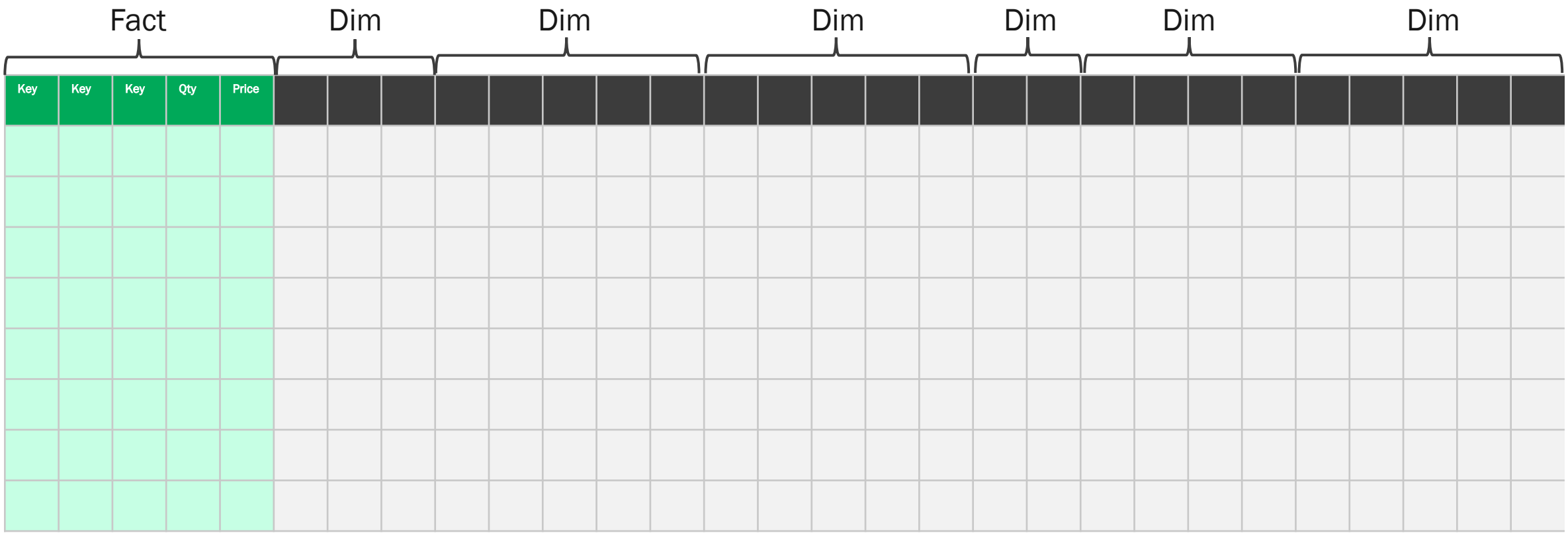
Propagates automatically if allowed by direction of relationship (RELATED)



Expanded Table
Is that you One Big Table?

Native column on Fact

Related column



Let’s dive into this. What are we filtering here?

```
CALCULATE (
    [Sales],
    FILTER (
        ‘Sales’,
        ‘Sales’[Unit Price] > 100
    )
)
```

Thus, filter columns not tables

Example of bad practice:

```
CALCULATE (
    [Sales],
    FILTER (
        ‘Sales’,
        ‘Sales’[Unit Price] > 100
    )
)
```

Results | Result 1 of 1 | Copy

	Sales[Product Key]	Sales[Order Date]	Sales[Ship Date Key]	Sales[Customer Key]	Sales[Sales Territory Key]	Sales[Order Quantity]	Sales[Unit Price]	Sales[Sales Amount]
1	483	10/29/2024 12:00:00 AM	20241111	13099	5	1	120	120
2	361	5/2/2025 12:00:00 AM	20250514	13145	2	1	2294.99	2294.99
3	323	7/28/2023 12:00:00 AM	20230809	15254	5	1	782.99	782.99
4	589	4/9/2025 12:00:00 AM	20250421	15254	5	1	769.49	769.49
5	343	12/27/2023 12:00:00 AM	20240108	15491	5	1	782.99	782.99
6	353	1/26/2025 12:00:00 AM	20250208	15491	5	1	2319.99	2319.99
7	606	10/17/2024 12:00:00 AM	20241030	16668	3	1	539.99	539.99
8	359	5/11/2025 12:00:00 AM	20250523	16916	5	1	2294.99	2294.99
9	580	11/17/2024 12:00:00 AM	20241130	17120	2	1	1700.99	1700.99
10	356	5/23/2024 12:00:00 AM	20240605	27281	5	1	2071.42	2071.42
11	574	4/13/2025 12:00:00 AM	20250425	27281	5	1	2384.07	2384.07
12	352	5/18/2024 12:00:00 AM	20240531	27286	3	1	2071.42	2071.42
13	360	5/3/2024 12:00:00 AM	20240516	27421	2	1	2049.1	2049.1
14	486	2/14/2025 12:00:00 AM	20250227	11241	7	1	159	159
15	568	11/7/2024 12:00:00 AM	20241120	11241	7	1	742.35	742.35
16	360	3/8/2024 12:00:00 AM	20240321	11241	7	1	2049.1	2049.1
17	591	9/30/2024 12:00:00 AM	20241013	11241	7	1	564.99	564.99
18	566	5/16/2025 12:00:00 AM	20250528	11242	7	1	742.35	742.35
19	360	3/17/2024 12:00:00 AM	20240330	11242	7	1	2049.1	2049.1

8 columns, 16.369 rows

Possible optimization:

```
CALCULATE (
    [Sales],
    KEEPFILTERS ( ‘Sales’[Unit Price] > 100 )
)
```

	Sales[Unit Price]
1	3399.99
2	2384.07
3	2319.99
4	3374.99
5	539.99
6	2294.99
7	159
8	742.35
9	120
10	769.49
11	3578.27
12	2443.35
13	1214.85
14	2049.1
15	564.99
16	1700.99
17	2071.42
18	1120.49
19	2181.56
20	1000.44
21	699.1
22	782.99

1 column, 22 rows

Filter columns, not tables, in DAX (SQLBI)

Wrap up

DAX Pillars

⇶ Row Context

Row context iterates a table (it does not filter it natively).

When you add a column to a table, the row context lets a single row's calculation use other values *in the same row*.

It does not propagate automatically to other tables. If you want to propagate this, use RELATED or RELATEDTABLE.

A column reference in a DAX expression requires a row context to return the value of the column from a table.

Row Context is iterating "the visible" table row by row. We shall come back on who decides which are the visible rows.

The row context is native in Calculated Columns in DAX.

Filter can be injected in calculated columns through CALCULATE (context transition)

Filter Context

Filter context filters the data model

When you "ask for a table", the filter context is the set of filters that gets applied before the table is handed over to you.

Propagates automatically if allowed by direction of relationship

Report-, Page- and Visual Level Filters, slicers, visual interactions and within visual → Every data point is calculated within its own filter context

You can use the CALCULATE DAX function to modify filter context and then calculate a number under the new filter context with overwrite, remove or add

Filter context is evaluated first before calculating the value in a CALCULATE

A filter is a table. A table is a list of values. The filter context is the set of active filters.

Native in measures

Iterators

Through iterators, you can create a row context in a measure (iterate a table) or a nested row context in a calculated column
Iterations do not filter and filters do not iterate: Iterators iterate (create a row context) on the visible rows of a table.

Who decides which rows are visible so they can be iterated? The filter context.

So, the filter context filters the model first and sets the rows available for each calculation, the row context iterates those rows.

⇆ Context Transition

There is no filter context in a row context. Unless you specify a context transition.

Context transition is the process of converting a row context to an equivalent filter context

Use CALCULATE or a Measure Reference in a row-context to enable context transition.

Every measure reference always has an implicit CALCULATE surrounding it

Context transition is expensive operation. If used during an iteration on a table with 10 columns and one million rows, then CALCULATE needs to apply 10 filters, one million times.

Context transition does not only filter one row

Context transition uses columns that are not present in the formular

Calculated tables and columns aren't evaluated within filter context

Expanded Table

When you ask for a table, you get the expanded table

Looks like the One Big Table (OBT) as it joins the related table columns

If you refer to the fact table of Sales, it will refer to all native columns in the Fact table + all related columns from the dimension tables that have a one-to-many relationship to the fact table

For performance optimization, always focus on only including the needed columns. Example of bad practice:

```
SUMX (
    FILTER (
        'Fact Table',
         Expression
    )
)
```

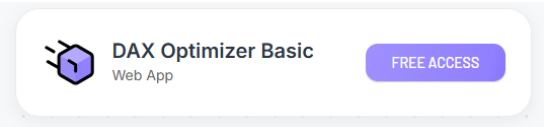
"Never filter tables, always filter columns"
- SQLBI

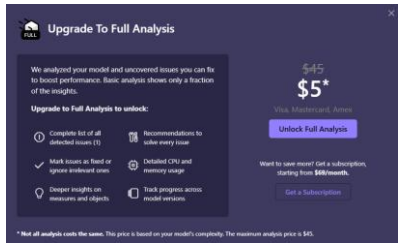
DAX Optimizer

”Marco Russo as a Service”

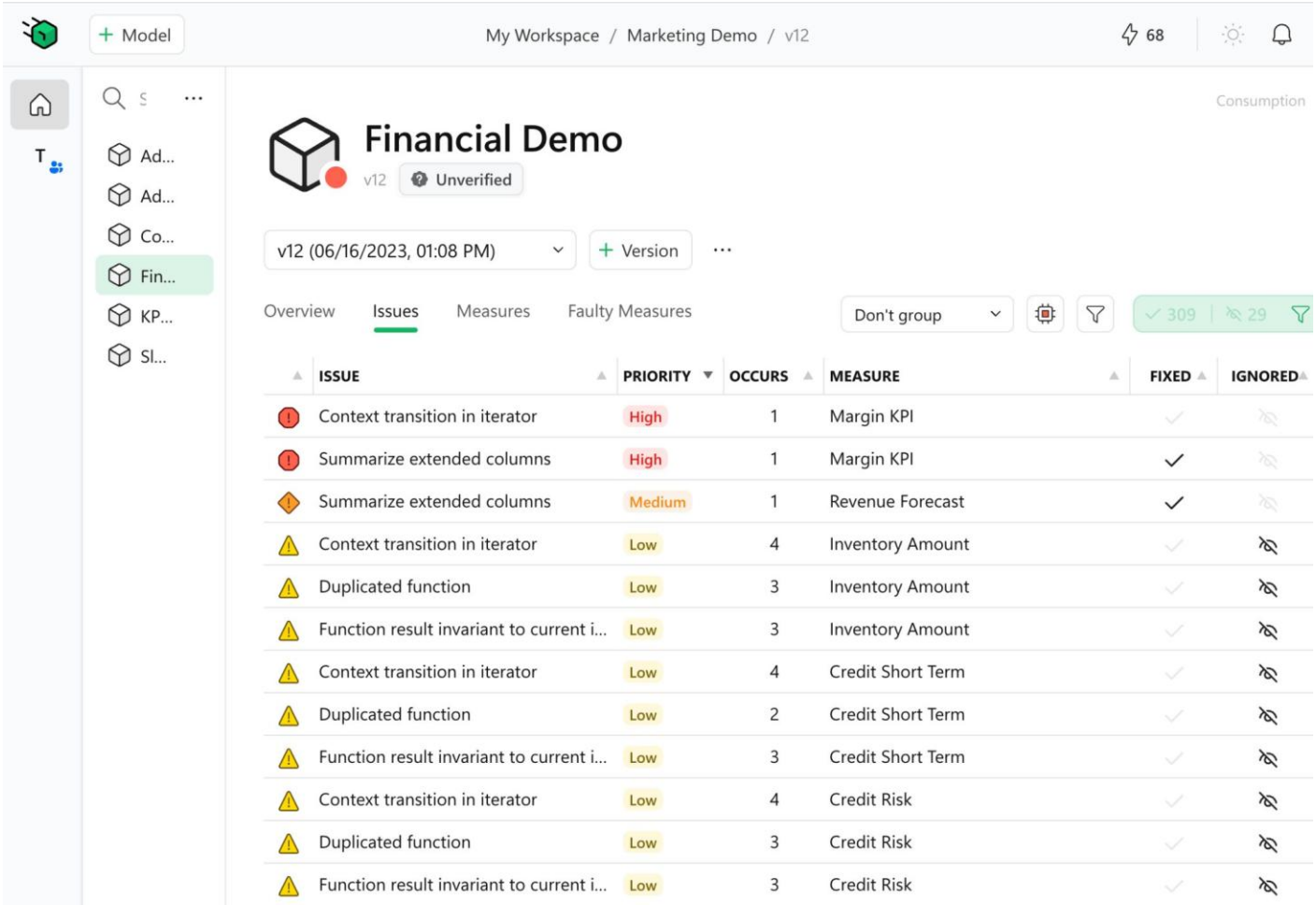
DAX Optimizer is a tool – delivered as a service – that helps identify potential **bottlenecks** and suggests impactful **solutions and priorities**.

You still write the code, but it is like **you have a mentor** on the side teaching you best practices, focusing on the most impactful area of your measures.





https://www.daxoptimizer.com/free/



Q&A